

kodama-cpp: Cross-Validated Accuracy Maximization on CPU, CUDA, and Apple Metal

Moussa Kassim^{1,2,*}

MOUSSA.KASSIM@ICGEB.ORG

Martin Ocharo^{1,2,*}

MARTIN.OCHARO@ICGEB.ORG

Dalia Ahmed¹

DALIA.AHMED@ICGEB.ORG

Dupe Ojo¹

DUPE.OJO@ICGEB.ORG

Alessia Vignoli^{3,4}

VIGNOLI@CERM.UNIFI.IT

Leonardo Tenori^{3,4,†}

TENORI@CERM.UNIFI.IT

Stefano Cacciatore^{1,2,†}

STEFANO.CACCIATORE@ICGEB.ORG

¹ *Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology (ICGEB), Cape Town 7925, South Africa*

² *Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*

³ *Department of Chemistry “Ugo Schiff”, University of Florence, Sesto Fiorentino, Italy*

⁴ *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*

* *Moussa Kassim and Martin Ocharo contributed equally.*

† *Leonardo Tenori and Stefano Cacciatore are co-corresponding authors.*

Editor: To be assigned

Abstract

KODAMA searches for latent structure by maximizing the cross-validated predictability of an evolving label vector. We present kodama-cpp and make four contributions. First, KODAMA is implemented as a standalone C++17 library with a typed, wrapper-independent API, allowing thin R and Python interfaces to share one numerical implementation. Second, native multicore CPU, NVIDIA CUDA, and Apple Metal backends implement float32 nearest-neighbor search, k-means, SIMPLS followed by latent-space LDA, PCA, graph operations, and reusable workspaces without silent CPU fallback. Third, the current search uses a common prediction-guided proposal and evaluation scaffold, while PLS-LDA adds a disclosed transition-coarsening move and fragmentation term to its state score. Proposal size decreases smoothly, an error-scaled temperature permits early exploration, and each cycle performs exactly one new cross-validation pass. Fourth, the landmark-projection strategy introduced in the KODAMA bioRxiv work is formalized for general matrix input through exact-quota stratified sampling, optimization on landmarks, and supervised projection to the remaining samples. These policies retain cross-validated predictability as the internal signal but need not reproduce historical stochastic trajectories. We validate backend identity, numerical behavior, runtime, memory, landmark-selection contracts, and external-label diagnostics separately so that systems acceleration is not conflated with the optimization objective.

Keywords: KODAMA, unsupervised learning, cross-validation, heterogeneous computing, CUDA, Metal, C++ library, nearest neighbors, PLS-LDA

1 Introduction

High-dimensional data analysis often begins with clustering or dimensionality reduction, but neither operation directly asks whether the discovered groups are predictable from the data. KODAMA takes a different view: a useful labeling of samples should be reproducible by a classifier under cross-validation. The labels are not treated as ground truth; they are optimization variables whose quality is measured by held-out prediction.

The original KODAMA paper introduced this idea as knowledge discovery by accuracy maximization, and the subsequent R package made it available as an unsupervised and semi-supervised feature-extraction tool for noisy high-dimensional data. The Bioinformatics package paper describes the algorithm as repeated random label initialization, iterative maximization of cross-validated accuracy by label swapping, and construction of a dissimilarity matrix from the resulting label vectors. The R documentation exposes the same logic through M independent iterative processes, Tcycle optimization steps, KNN or PLS-DA classifiers, and separate `KODAMA.matrix` and `KODAMA.visualization` functions.

The repeated classifier fits inside KODAMA create a systems problem that is not resolved by translating individual R expressions into C++. Neighbor indices, fold matrices, class encodings, latent projections, and evolving labels must be reused across many cross-validation calls without changing the accepted label trajectory. A useful accelerator implementation must therefore preserve the mathematical state machine while changing where that state lives and how it moves.

`kodama-cpp` addresses this problem with a standalone C++17 core and explicit CPU, CUDA, and Apple Metal backends. It currently supports two classifier families in the KODAMA optimization layer: KNN for local neighborhood consistency and PLS-LDA for low-rank linear discrimination. R and Python wrappers call the same typed API, so fold construction, proposals, acceptance decisions, and result metadata are not reimplemented by each language interface. Their public functions, defaults, accepted choices, one-based best-run convention, and result fields are matched; Python only transliterates dotted R argument names to `snake_case`.

The manuscript is organized around four novelties. The first is software ownership: KODAMA becomes a standalone C++ library from which R and Python wrappers can be built without reimplementing folds, proposals, classifiers, or graph construction. The second is heterogeneous execution through native CUDA and Apple Metal backends under the same float32 result contract. The third is a guided label-evolution strategy that uses out-of-fold predictions, class-transition evidence, adaptive proposal sizes, degeneracy guards, and error-scaled cooling while preserving one new CV evaluation per cycle. The fourth is a general landmark-projection procedure, first introduced in the KODAMA bioRxiv study, that makes the expensive optimization depend on a representative subset and then projects each optimized solution to all samples.

We deliberately frame this contribution as a software-methods paper, not as a claim that KODAMA replaces clustering, semi-supervised learning, or manifold learning. Cross-validated accuracy is the internal optimization criterion used to construct candidate label vectors. External labels, when available, are used only for diagnostics such as adjusted Rand index, silhouette, local purity, and runtime-quality tradeoffs.

Table 1: Main contributions of kodama-cpp.

Contribution	Role in kodama-cpp
Standalone C++ core	Moves folds, classifiers, label evolution, projection, and graph construction into one C++17 library so that R and Python wrappers share the same typed implementation.
CUDA and Metal backends	Provides native float32 KNN, k-means, SIMPLS-LDA, PCA, and reusable accelerator state with strict backend identity and no silent CPU fallback.
Guided CV maximization	Uses a common prediction-guided proposal/evaluation scaffold; PLS-LDA additionally applies transition coarsening and a fragmentation-aware state-score term.
Landmark projection	Formalizes the bioRxiv landmark idea as exact-quota stratified sampling, optimization on a representative subset, supervised projection, and ensemble graph correction.

2 KODAMA objective

Let X in $\mathbb{R}^{(n \times p)}$ denote the input matrix, c in $\{1, \dots, K\}^n$ a candidate label vector, F a classifier family, and Π a fold assignment. For each validation sample i , the classifier is trained on samples not in $\Pi(i)$ and predicts c_i from x_i . The empirical objective is the held-out accuracy $A(c; X, F, \Pi)$. KODAMA searches for a labeling c^* that maximizes this quantity, turning unsupervised structure discovery into a self-guided supervised prediction problem.

One KODAMA run starts from an initial partition. In the historical R interface, this can be one label per sample or a clustering-derived vector W ; a fraction of samples can be selected for each run, with 75% as the documented default. The current C++ interface keeps the same idea but exposes starting labels, splitting, landmarks, and fixed or constrained groups as typed inputs. During each cycle, misclassified samples or constrained groups generate candidate label moves. A move is accepted if it improves the objective, or under a temperature rule that allows exploration early in the run. The landmark-projection principle introduced by Abdel-Shafy et al. (2025) is treated here as a general matrix algorithm: label optimization is performed on selected landmarks and the chosen classifier restores a complete label vector before ensemble construction.

The M runs are independent. This independence is important both statistically, because it averages over random initialization and proposal order, and computationally, because backend schedulers can process runs without changing the objective. The final KODAMA representation is obtained from the ensemble of optimized label vectors: pairs of samples that repeatedly receive compatible labels are pulled closer in the derived graph or dissimilarity, while unstable pairs are weakened.

$$c^* = \arg \max_c A(c; X, F, \Pi), \quad (1)$$

where A is the held-out accuracy of classifier family F under fold assignment Π .

2.1 Landmark-projection strategy

The landmark-projection principle was introduced in the KODAMA bioRxiv study (Abdel-Shafy et al., 2025) to reduce the cost of repeated accuracy maximization: optimize labels only on a selected subset, fit a supervised model to each optimized landmark vector, and predict labels for the remaining samples before constructing the ensemble dissimilarity. kodama-cpp isolates this principle as a general algorithmic component that does not depend on a particular application domain or wrapper language.

The current selector replaces k-means with one center per requested landmark by a coarse partition followed by exact population-proportional allocation. For effective landmark count L and coarse stratum c containing n_c of n samples, $q_c = L \cdot n_c / n$ is the expected allocation. The algorithm samples $\text{floor}(q_c)$ rows without replacement, then assigns the remaining slots by randomized systematic rounding of the fractional quotas. It therefore returns exactly L distinct landmarks and satisfies $E[L_c] = L \cdot n_c / n$ without requiring L k-means centers.

Sampling strata and labels being optimized are intentionally different objects. After landmarks are selected, a separate expression-space partition initializes the candidate labels. KODAMA evolves those labels for Tcycle steps, stores the selected vector from each of M independent runs, and projects it to non-landmarks with the selected KNN or PLS-LDA classifier. The final graph is then corrected from agreement across complete projected vectors, so non-landmarks contribute to the returned representation even though the iterative CV search is performed on the landmark subset.

Table 2: Validation of the landmark-projection implementation.

Validation dimension	Current manuscript evidence
Selection contract	Unit tests require the exact effective landmark count, distinct sampled rows, fixed-seed repeatability, and valid represented/occupied-stratum diagnostics.
Backend contract	CPU and CUDA matrix runs expose matching landmark counts and diagnostics; selection remains host-orchestrated and independent of classifier backend.
Scalability isolation	On flow18, selecting 750,016 landmarks from 300 coarse strata required 0.321 seconds; the separately measured 100-neighbor global graph dominated the complete call.
Quality evaluation	Landmark tests report downstream CV accuracy, ensemble stability, and external diagnostics separately from selection time so representativeness is not inferred from speed alone.

2.2 Label-vector search mechanics

The object optimized by KODAMA is the label vector itself. At any time the algorithm holds a candidate vector y on the landmark samples. A classifier is not used to predict an external truth; instead it is trained to reproduce y under cross-validation. A label vector is therefore good when the labels assigned to held-out samples are predictable from the measured variables. The raw accuracy attached to a candidate, $\text{acc}(y)$, is the fraction of

landmark samples for which the held-out prediction $\text{pred}_i(y)$ equals the candidate label y_i .

The first CV pass is run on the initial label vector. This gives both an initial accuracy and a prediction vector. The prediction vector is then used as a proposal guide. If a sample or constrained group repeatedly receives a different predicted label from the classifier, that predicted label is evidence that the current label vector is not aligned with the structure that the classifier can reproduce. KODAMA uses this information to propose a new label vector rather than drawing completely blind random swaps.

Within a cycle, the algorithm chooses a proposal base. In evolutionary mode this is the current accepted vector; otherwise it is the best vector found so far. For one-based cycle t , it sets $p_t = t / (T_{\text{cycle}} + 1)$, $s_t = p_t^2(3 - 2p_t)$, and $q_{\text{max}} = 1 + \text{floor}((G - 1)(1 - s_t))$, then samples q uniformly from $1, \dots, q_{\text{max}}$. Early cycles can therefore alter many groups and later cycles fewer. For each sampled group, the replacement label is drawn from the empirical distribution of the previous CV predictions among its non-fixed members.

The class-transition step aggregates the same information at label level. It builds a transition matrix N_{ab} counting samples whose current label is a and whose held-out prediction is b . Large off-diagonal counts indicate that the classifier consistently maps one candidate class into another. KODAMA may therefore propose merging unstable source labels into predicted destination labels, while rejecting moves that collapse the solution to a single class or do not reduce unstable fragmentation.

The many-to-one transition proposal uses the same matrix N . For a source class a and a destination b , the expected count under independence is $n_a n_b / n$. A candidate absorption is considered only when N_{ab} exceeds this expectation and also exceeds the number of samples that remain in class a . Candidate destinations are sampled with probability proportional to their positive surplus, and the selected source classes are mapped to the destination only if the move reduces the number of active classes without collapsing all labels into one class.

Under the standard KODAMA.matrix policy, the PLS-LDA path applies an additional automatic coarsening proposal before the CV evaluation. It computes the class entropy $H = -\sum_k p_k \log p_k$ and the effective number of classes $K_{\text{eff}} = \exp(H)$. Fragmentation is measured as $\max(0, \log(K / \max(1, K_{\text{eff}})))$. Classes with high transition instability and small movable size are proposed for merging into their strongest predicted destination; the merge budget is $\text{ceil}(T_{\text{frag}} \max(0, K - K_{\text{eff}}))$, where $T_{\text{frag}} = \text{fragmentation} / (\text{fragmentation} + \text{weighted_transition_entropy} + 1)$. This keeps the proposal tied to CV confusion rather than external labels. The branch is PLS-LDA-specific because latent LDA estimates a q -dimensional mean for every active class and a pooled covariance from the remaining degrees of freedom; many small classes therefore affect a global fitted model. KNN voting has no analogous global per-class parameter fit. The lower-level CoreOptions interface exposes the coarsening and diversity switches for controlled studies.

After these proposal moves, the proposed vector is evaluated by exactly one new CV pass. The classifier-common score is $S_0 = A \sqrt{1 - \sum_k p_k^2}$, where A is raw CV accuracy and p_k is the class proportion. KNN uses $S_{\text{KNN}} = S_0$. PLS-LDA uses $S_{\text{PLS-LDA}} = S_0 - (1 - A) \max(H / \log n, \text{fragmentation} / (1 + \text{fragmentation}))$ when automatic coarsening is active. The labels stored in `clbest` maximize the applicable classifier-specific

score, and accbest is the raw A evaluated for exactly those labels. Thus the CV objective and evaluation count are common, but the state-selection policy is not identical.

The best vector is updated whenever the proposal score improves. In evolutionary mode, a separate current vector is also maintained. A worse proposal can become current with probability $\exp((\text{score_new} - \text{score_current}) / \text{tau_t})$, where, for one-based cycle t , $\text{tau_t} = \max(1e-9, 0.10 \max(0, 1 - \text{current_accuracy}) (1 - t/T_{\text{cycle}}))$. The $1e-9$ floor makes the terminal cycle numerically defined and effectively greedy. Repeating this process for M independent seeds gives an ensemble of guarded high-predictability label vectors rather than relying on a single local optimum.

2.3 Raw accuracy, acceptance score, and diagnostics

It is useful to distinguish three quantities that appear in the implementation. The first is the raw held-out CV accuracy $A(y)$, which is the classifier reproducibility score reported for a label vector. The second is the proposal acceptance score $S(y)$, which is the stochastic-search objective actually used to choose moves when degeneracy guards are enabled. The third is an external diagnostic, such as ARI or embedding compactness, which is never optimized by KODAMA but is reported when reference labels are available.

This distinction is necessary because a high raw CV accuracy alone can be achieved by a collapsed or overly coarse label vector. Such a vector may be easy for the classifier to reproduce but poor as a representation of latent structure. The diversity factor is common to KNN and PLS-LDA. Transition coarsening and the fragmentation subtraction are PLS-LDA-specific standard policies; they change which proposals are selected, while the reported accuracy remains the raw held-out classifier accuracy.

The fixed- $M = 100$ sensitivity study provides empirical motivation, not a causal ablation. At $T_{\text{cycle}} = 100$, median active-class counts were 9 for KNN versus 24 for PLS-LDA on MetRef and 32 versus 71 on USPS, showing that the classifier paths occupy different fragmentation regimes despite high held-out accuracy. The archived experiment did not toggle only the PLS-LDA penalty, so the manuscript does not claim that this term universally improves ARI, silhouette, or runtime. It is reported as a classifier-specific implementation policy whose isolated benefit remains to be tested.

2.4 Reproducible KODAMA.matrix procedure

The public KODAMA.matrix routine is reproducible from the following option definitions and run-level procedure. The description names the C++ options because the R and Python wrappers should expose the same contract.

2.4.1 RUN-LEVEL OPTIMIZATION RULE

1. Copy the input matrix once into float32 working storage. Before the M loop, build one full-data neighbor graph G_0 for label projection and final agreement correction. When visualization initialization is requested, run one backend-native float32 PCA on the same working matrix and derive the separately scaled UMAP and openTSNE starts from those scores. The graph and both starts are returned with the matrix result.

Table 3: KODAMA.matrix options that determine the optimization.

Option	Role in the algorithm
M / runs	Number of independent KODAMA runs. KODAMAMatrixOptions::runs defaults to 100. Run r uses seed $\text{seed} + r$ and writes one length-n label vector to the result matrix.
Tcycle / cycles	Number of proposal/evaluation cycles inside each run. KODAMAMatrixOptions::cycles defaults to 20 for compatibility with the historical interface; the benchmark-quality setting used in this manuscript is 100. CoreOptions accepts the same role when the core optimizer is called directly.
landmarks	Exact number of samples optimized directly in one run after validation. The default is 10000; if the requested value is at least n, the implementation uses $\text{ceil}(0.75 \text{ } n)$, then caps the value to $[2, n-1]$. Quota sampling always returns this effective count.
splitting	Initial number of label classes and matrix-input landmark strata. If not supplied, splitting is 100 for $n < 40000$ and 300 otherwise. After landmark sampling, a separate matrix-space k-means initializes labels on the selected rows so sampling strata remain distinct from labels being optimized.
constrain	Optional group vector. Empty means one movable group per sample. Non-empty constraints keep all members of a group in the same CV fold and make the group the unit of label proposal.
fixed	Optional binary vector. Fixed samples are included in CV but are not relabeled by proposal moves.
graph_neighbors	Number of neighbors retained in the final sparse graph. The actual k is $\min(\text{graph_neighbors}, \text{landmarks}, \text{floor}(0.75 \text{ } n - 1))$ after enforcing $k \geq 1$.

- For each run $r = 1, \dots, M$, create a random generator with seed $\text{seed} + r$. Let L be the validated landmark count. Run k-means on all rows with splitting centers and 10 iterations to obtain coarse matrix-derived sampling strata.
- For each stratum c, compute $q_c = L \text{ } n_c / n$, take $\text{floor}(q_c)$ samples without replacement, and assign the remaining slots by randomized systematic rounding of $q_c - \text{floor}(q_c)$. This returns exactly L distinct landmarks. If starting labels are supplied, subset them and apply constrained-majority relabeling when needed. Otherwise run matrix-space k-means on the landmarks with $\text{init_k} = \max(2, \min(\text{splitting}, L))$, independently of the sampling partition.
- Build CV folds on the landmark set. With no constrain vector, samples are assigned to folds directly. With a constrain vector, each group is assigned as a whole; stratified folds use the group-majority label and non-stratified folds shuffle groups. KODAMA.matrix currently sets the internal CV folds to non-stratified for both KNN and PLS-LDA core paths.
- At cycle t, choose the proposal base. In evolutionary-chain mode, the base label vector and prediction vector are the current accepted state; otherwise they are the current best state. These carried predictions drive the label proposal, and the proposed label vector is then evaluated by one CV pass.

6. Sample proposal groups. Let G be the number of groups and index cycles by $t = 1, \dots, T_{\text{cycle}}$. With adaptive proposal size enabled, $p_t = t / (T_{\text{cycle}} + 1)$, $s_t = p_t^2 (3 - 2 p_t)$, $\theta_t = 1 - s_t$, and $q_{\text{max}}(t) = 1 + \text{floor}((G - 1) \theta_t)$. Draw q uniformly from $\{1, \dots, q_{\text{max}}(t)\}$ and sample q groups without replacement. Without adaptive proposal size, q is uniform on $\{1, \dots, G\}$.
7. For each sampled group, collect the non-fixed members. Their candidate replacement labels are the previous CV predictions for those members. The replacement label is drawn from the empirical frequency of those predicted labels, and all eligible members of the group are relabeled together.
8. Apply the classifier-common class-transition proposal. The transition matrix has entries $N_{ab} = \#\{i: \text{current label } a, \text{ CV prediction } b\}$. The many-to-one proposal requires $N_{ab} - n_a n_b / n > 0$ and $N_{ab} > N_{aa}$, prefers destinations supported by multiple source classes, and samples among eligible destinations in proportion to summed positive surplus. It rejects moves that collapse to one class or fail to reduce class count. Only for PLS-LDA, an additional coarsening move may merge small unstable classes according to the same transition matrix.
9. Evaluate the proposed labels with exactly one CV pass. Let A_t be the fraction of landmark samples whose proposed label equals the held-out CV prediction. Both classifiers multiply A_t by $\sqrt{1 - \sum_k p_k^2}$, where p_k is the current class proportion. Only the PLS-LDA policy subtracts the additional entropy/effective-class fragmentation term when class coarsening is enabled. Record raw A_t alongside the selected vector.
10. Update the best vector whenever the proposal score improves. In evolutionary-chain mode, also update the current vector if the score improves the current score, or with probability $\exp((\text{score} - \text{current_score}) / \tau_t)$, where $\tau_t = \max(1e-9, 0.10 \max(0, 1 - \text{current_accuracy}) (1 - t / T_{\text{cycle}}))$.
11. After T_{cycle} cycles, project the optimized landmark labels back to all samples. For the KNN path, each non-landmark receives the majority label among its labeled landmark neighbors in the global graph. For the PLS-LDA path, the final PLS-LDA model predicts the non-landmarks. If constraints are present, the final labels are replaced by the majority label inside each constraint group.

2.4.2 FINAL GRAPH AND DISSIMILARITY

The C++ implementation stores the final KODAMA representation as a sparse neighbor graph rather than materializing a dense n by n matrix by default. KODAMAGraph constructs one global KNN graph using the selected metric and `graph_neighbors` and computes backend-matched PCA starts. KODAMAGraphResult owns only indices, float32 distances, two initialization arrays, dimensions, backend provenance, and timings; it does not own, alias, or copy the source MatrixView. KODAMAMatrix can consume that result alone or alongside a separately supplied MatrixView; raw-matrix input invokes the same preparation logic internally. The graph owner is kept through all independent runs, compacted in place,

and moved into the result. The default applies agreement correction to that same storage. If correction is deferred, the result retains the base graph and `KODAMADissimilarityInPlace` can correct it lazily; no simultaneous base_knn and corrected-knn copies are retained.

Let c_i^r be the final label of sample i from run r . For each retained edge (i,j) with original nonnegative distance d_{ij} , compute V_{ij} as the number of runs where both endpoint labels are nonzero, and S_{ij} as the number of those valid runs where the two labels agree. If $V_{ij} = 0$ or $S_{ij} = 0$, the corrected edge distance is set to infinity. Otherwise $a_{ij} = S_{ij} / V_{ij}$ and $d'_{ij} = (1 + d_{ij}) / a_{ij}^2$. Each neighbor row is then sorted by d'_{ij} . This implementation contract is not invariant to an arbitrary rescaling of d , so matched comparisons fix preprocessing and metric. The resulting graph is the KODAMA-corrected graph used by `KODAMA.visualization`.

A dense dissimilarity can be derived from the same agreement statistic by evaluating the formula for all sample pairs. The library keeps the sparse graph form because downstream UMAP, openTSNE, and graph clustering only need local neighborhoods, and the sparse form is the scalable object shared by the available backends.

2.5 Main algorithm pseudocode

The compact algorithm below connects landmark selection, guided label evolution, projection, and agreement-graph construction. Each inner cycle contributes exactly one new CV evaluation; the initial state contributes one baseline CV evaluation.

Algorithm: KODAMA with guided evolution and landmark projection

```

Input: X or prepared graph P, classifier F, runs M, cycles T, landmarks L
      splitting K0, folds Pi, seed s
if P is absent then P <- KODAMA.Graph(X)
G0 <- P.graph; (Z_U, Z_T) <- P.PCA_starts
for r = 1,...,M do
  strata <- CoarsePartition(X, K0, seed=s+r)
  L_r <- ExactQuotaSample(strata, L, seed=s+r)
  y <- InitialPartition(X[L_r,], K0, seed=s+r)
  (A, pred) <- CrossValidate(F, X[L_r,], y, Pi_r)
  (y_best, A_best, S_best) <- (y, A, Score_F(y, A))
  for t = 1,...,T do
    y_prop <- GuidedProposal(y, pred, t, T, constraints)
    if F == PLS-LDA then y_prop <- TransitionCoarsen(y_prop, pred)
    (A_prop, pred_prop) <- CrossValidate(F, X[L_r,], y_prop, Pi_r)
    update (y_best, A_best, S_best) when Score_F(y_prop, A_prop) improves
    (y, pred, A) <- AcceptOrCool(y_prop, pred_prop, A_prop, t, T)
  end for
  c[,r] <- ProjectLabels(F, X[L_r,], y_best, X)
end for
for each edge (i,j) in G0 do
  V_ij <- {r : c[i,r] and c[j,r] are valid}; a_ij <- mean over r in V_ij of 1[c[i,r] = c[j,r]]
  d_ij' <- infinity if a_ij = 0 else (1 + d_ij) / a_ij^2
end for
return corrected graph G, projected label matrix c, and paired raw CV accuracies

```

3 Implementation architecture

The core library exposes C++ functions rather than wrapper-specific entry points. A `MatrixView` accepts double or float inputs, but all analysis paths convert to and retain float32 matrices and workspaces. The core owns fold assignments, compact class encodings, current

and best label vectors, classifier buffers, and typed result metadata. KODAMAGraphResult owns only graph arrays, two PCA starts, and metadata; it never owns or aliases the source MatrixView. The public API is grouped into cross-validation kernels, core label optimization, graph preparation, KODAMA matrix construction, PCA, and embedding utilities.

Backend selection is strict and observable. CPU, CUDA, and Metal entry points report the backend that actually executed, and an unavailable accelerator raises an error. This rule is essential for reproducible benchmarking because a nominal GPU call that silently falls back to CPU would preserve outputs while invalidating performance claims.

KNN kernels have backend-specific implementations under one voting contract. The dependency-light CPU path uses a package-owned float32 HNSW implementation with a deterministic connected base seed before per-node-locked parallel insertion, batched candidate distance evaluation, contiguous graph storage, and parallel batched queries. CUDA provides package-owned exact and recall-tuned IVF-Flat search plus GPU k-means using CUDA Toolkit libraries. Apple Metal provides native exact and recall-tuned IVF-Flat search plus GPU k-means using Metal compute kernels. Neighbor structures are reused where the mathematics permits, avoiding repeated wrapper-level index construction.

PLS-LDA uses a SIMPLS strategy following the fastPLS implementation rather than a simplified SVD approximation. CPU, CUDA, and Metal paths operate on float32 matrices and compute class-label cross-products directly instead of constructing dense one-hot response matrices where possible. The requested component count is treated as the evaluated component count whenever mathematically feasible, rather than being replaced by an internal model-selection shortcut. The library exposes no PLS-cKNN classifier; KODAMA optimization is limited to KNN and PLS-LDA.

CUDA and Metal KODAMA calls keep one full-data graph resident through all M runs and allocate one persistent scratch lane per concurrent worker. Each lane retains landmark and fold layouts, compact labels, projections, KNN votes, cross-products, PLS weights, and LDA workspaces. A new M run changes landmark membership and fold contents, so those values are refreshed once in the existing buffers; Tcycle proposals overwrite only label-dependent state. This is buffer and data-lifecycle residency, while stochastic proposal decisions remain host orchestrated.

Latent-space LDA is fitted from sufficient statistics. For training scores T , class counts n_c , class means μ_c , q retained components, and C active classes, the pooled within-class covariance is $[T' T - \sum_c n_c \mu_c \mu_c'] / \max(1, n - C)$. Each discriminant direction solves $\Sigma w_c = \mu_c$ by Cholesky factorization and triangular substitution; Σ is never explicitly inverted. The diagonal regularizer is $\lambda = \rho s$, where $s = \text{trace}(\Sigma) / q$ when finite and positive and $s = 1$ otherwise. The deterministic sequence ρ in $\{1e-8, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2\}$ is advanced only when Cholesky fails. Prediction retains the ordinary LDA discriminant $t' w_c - 0.5 \mu_c' w_c + \log(n_c / n)$; the sequence is a numerical safeguard, not a tuned parameter.

The library also exposes float32 randomized PCA as a preprocessing and embedding-initialization primitive. A Gaussian range sketch, backend-specific automatic oversampling and power-iteration policy, orthonormal basis, and compact eigensolve produce scores, loadings, singular values, and explained variance. CPU, CUDA, and Metal entry points share this contract. The implementation follows the current fastEmbedR/fastPLS strategy but is standalone and does not link either package, Armadillo, Eigen, or RAFT.

UMAP and openTSNE are direct standalone ports of pinned fastEmbedR revision ef064f2a. UMAP builds fuzzy graphs by default, with binary weighting available explicitly; both are constructed directly in float32 CSR form, including the smooth-kNN bandwidth calculation and epochs-per-sample schedule; openTSNE uses the matching sparse affinities and exact or FFT-grid repulsion. The current openTSNE port also retains persistent CPU workers, cached FFT plans, parallel FFT preparation, and chunked CUDA graph launches. The CUDA path reuses pooled workspaces, and neither path links fastEmbedR or R.

Visualization initialization is part of the typed backend contract. One backend-native float32 PCA produces both starts from the raw matrix: openTSNE centers and scales scores to maximum component standard deviation 1e-4, while UMAP centers, max-absolute scales to 10, adds seeded float32 jitter, and recenters. Explicit coordinates take precedence; otherwise wrappers use explicit raw data, then a stored start only when its CPU/CUDA backend matches, and finally the reported graph-spectral UMAP or random openTSNE fallback. In a controlled CPU openTSNE comparison with matched neighbors and settings, fastEmbedR and kodama-cpp coordinates were identical. CPU UMAP is algorithmically matched but not asserted bitwise because kodama-cpp retains float32 iterative state whereas the compared R entry point stores its embedding in a double matrix.

KODAMA.graph prepares one full-data neighbor graph and backend-matched UMAP and openTSNE PCA starts without retaining the raw matrix. Four input forms share one dispatch contract: raw data; a prepared graph result; that result plus a separately supplied raw matrix; or a bare neighbor graph. Prepared and bare graph inputs avoid rebuilding neighbor search. With explicit raw features, landmark geometry and PLS-LDA use the original float32 variables; without them, the graph is converted into a PLS-compatible float32 representation with the standard self-tuning normalized Laplacian transform. Fixed-graph KNN reuses supplied neighborhoods and is not asserted to reproduce the stochastic trajectory of data-native KNN.

The graph-only PLS-LDA transform is deterministic conditional on the supplied seed. For row i , $\sigma_{i,i}$ is the median finite neighbor distance, falling back to the row mean and then to 1e-6 when needed. Each directed edge has weight $w_{ij} = \exp(-d_{ij}^2 / \max(1e-12, \sigma_{i,i} \sigma_{j,j}))$; reciprocal edges are added by taking the larger weight, and the operator is symmetrically normalized as $D^{-1/2} W D^{-1/2}$. Random Gaussian feature columns are repeatedly multiplied by this sparse operator, orthonormalized for at least eight iterations, and finally standardized before entering the ordinary PLS-LDA core.

The implementation deliberately keeps classifier paths clean. KODAMA optimization is exposed as KNN or PLS-LDA; auxiliary cross-validation kernels exist for testing and benchmarking, but the KODAMA optimizer is not a collection of benchmark-specific branches. The graph, embedding, and clustering utilities are similarly separated from the label-evolution loop.

3.1 Implementation novelty

kodama-cpp preserves cross-validated predictability as the KODAMA signal but contributes both a systems redesign and an explicit current search policy. The original R implementation remains the historical reference for accuracy maximization and ensemble construction.

kodama-cpp architecture

Standalone C++17 core with dependency-light CPU/Metal and opt-in CUDA backends

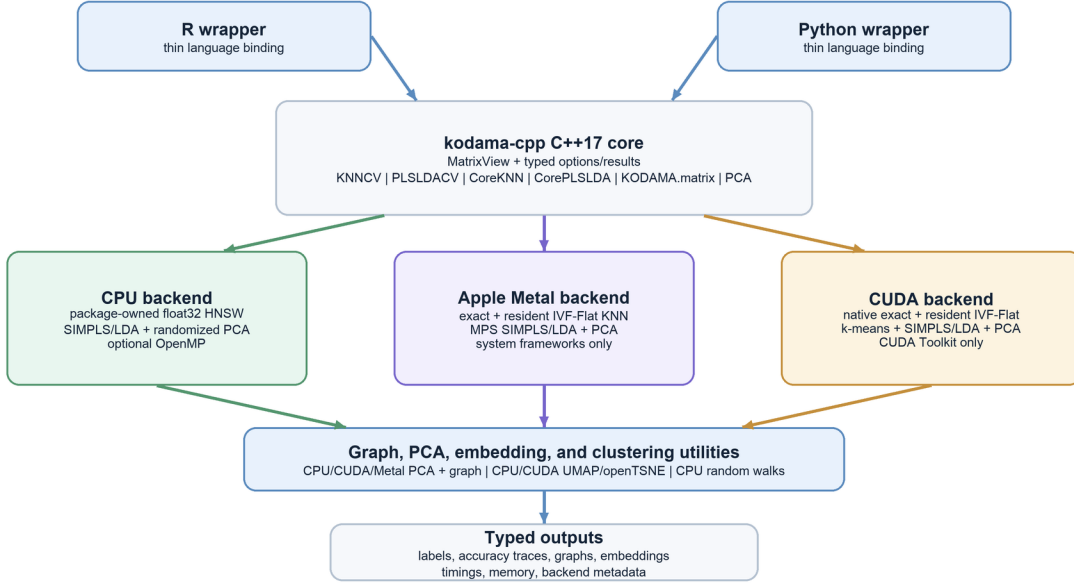


Figure 1: Architecture of kodama-cpp. The C++17 core owns the numerical kernels, CPU, CUDA, and Apple Metal backends, graph utilities, and typed outputs; R and Python remain thin wrappers.

In the new library, numerical state previously recreated across wrapper calls is represented once in typed C++ objects and reused across folds, proposal cycles, and independent runs.

This distinction matters because the runtime of KODAMA is the product of several nested operations: M independent searches, T_{cycle} proposal steps, cross-validation folds, and a classifier fit or vote inside each fold. Accelerating only one matrix multiplication leaves allocation, format conversion, index construction, and host-device traffic in the dominant loop. The implementation therefore optimizes the lifecycle of the complete CV state while retaining one objective evaluation per proposal.

The backends share mathematical invariants but not source-level execution strategies. KNN uses the same metric and voting rules with different search engines. PLS-LDA uses the same SIMPLS deflation and latent LDA model while each accelerator maps the dominant operations to its own memory and matrix-compute model. This separation avoids benchmark-specific branches inside the classifier and makes CPU/accelerator parity an explicit test target.

Guided evolution is deliberately implemented above the classifier layer. Previous held-out predictions determine sample/group proposals and the class-transition graph; the adaptive schedule determines how many groups may move; and each proposal receives one new CV pass. This is a common scaffold, not a classifier-independent state policy: KNN uses the diversity-guarded score directly, whereas PLS-LDA may add transition coarsening and

a fragmentation subtraction before the shared cooling decision. The policy can change stochastic trajectories relative to historical KODAMA without changing the definition of a CV prediction.

3.2 CUDA backend

The CUDA path is package-owned and uses only CUDA Toolkit libraries. Input matrices are materialized as contiguous float32 buffers. Exact KNN evaluates the requested metric directly, whereas IVF-Flat builds GPU k-means centroids and inverted lists and selects nprobe from an exact pilot query when recall tuning is enabled. The selected nlist, nprobe, pilot recall, and device id are returned with the result. Exact Lloyd assignment uses a bounded 256 MiB score workspace per independent M lane and batches rows when necessary; every row still evaluates every centroid with the same distance and tie rules. Assignments, centroid sums and counts, empty-cluster repair, list counts, prefix offsets, and row-ID scatter remain on CUDA. A move-only ResidentIVFIndex can retain the input, projection, centroids, offsets, and IDs for subsequent self or external-query calls without a hidden process-global cache.

For PLS-LDA, class labels are encoded compactly and the centered X'Y cross-product is formed from class sums on the GPU, avoiding a dense n-by-K one-hot response. Float32 SIMPLS uses reusable thread-local device workspaces that grow to the required capacity. Fixed-order class accumulation is followed by separate column-sum and centering kernels, avoiding a read/write race in X'Y centering. Each stream owns a persistent cuBLAS workspace. Separate CUDA kernels compute latent projections, class sums, covariance terms, and LDA scores, while fold caches retain train/validation layouts.

The default rank-one randomized SIMPLS refresh uses two power iterations. CUDA dispatches the resulting rank-one products as matrix-vector operations and omits the algebraically redundant 1-by-1 block eigensolve. After the first component, the preceding weight is the defined warm start, so the implementation also omits generation of a random vector that would be overwritten before use; this removes launches without changing any SIMPLS value. The multi-vector refresh remains available under the same deflation equations. Latent LDA forms pooled covariance from score and class-sum statistics, retries the trace-normalized ridge sequence only after a failed Cholesky factorization, and copies back predicted codes rather than score matrices.

The CUDA backend uploads the full graph once and keeps per-worker landmark/fold, label, projection, voting, SIMPLS, and LDA allocations alive across independent runs. Values that mathematically change are refreshed in place, and independent runs retain their seeds and proposal trajectories. Landmark selection, proposal decisions, and compact result collection remain host orchestrated, so the manuscript does not claim a fully device-side evolutionary state machine.

3.3 Apple Metal backend

The Apple backend is implemented in Objective-C++ on Metal rather than emulating CUDA. A persistent Metal state owns the device, command queue, and compute pipelines. Native kernels implement exact KNN, projected IVF-Flat search, Lloyd k-means assignment and centroid updates, empty-cluster repair, and inverted-list count, prefix, and scatter

operations. The same move-only ResidentIVFIndex contract retains Metal input and index buffers across search calls; self-search therefore does not upload the training matrix again.

The Metal PLS-LDA path uses float32 class-label cross-products and the same SIMPLS component limit as CPU and CUDA. Metal Performance Shaders execute the dominant Xw and X^t matrix products through MPSMatrixMultiplication. The same two-iteration incremental refresh and SIMPLS loading deflation are used on all three backends; the comparatively small response-space products and deflation updates remain host operations. This division is stated explicitly because a native Metal backend does not imply that every scalar operation is a GPU kernel.

Metal is a first-class backend contract: it has named KNNCV, PLSLDACV, CoreKNN, CorePLSLDA, and KODAMAMatrix entry points, reports Metal in result metadata, and is covered by dedicated parity tests. Unsupported Metal operations fail explicitly instead of being relabeled CPU execution. The R integration layer accepts `backend = 'metal'` explicitly and links the Metal, MetalPerformanceShaders, and Foundation system frameworks without FAISS or cuVS link flags. This makes Apple GPU results auditable in the same manner as CUDA results.

3.4 Shared accelerator contract

Table 4: Backend-specific execution under a shared KODAMA contract.

Invariant	CUDA realization	Metal realization
Internal precision	Contiguous float32 host/device matrices and reusable CUDA buffers.	Float32 shared MTLBuffers and MPSMatrix objects.
KNN	Exact or recall-tuned IVF-Flat with device-built lists, returned metadata, and an explicit resident-index owner.	Exact or recall-tuned IVF-Flat with device-built lists and the same resident-index lifetime.
PLS-LDA	GPU label cross-products, SIMPLS workspace, projection, class summaries, and LDA scoring.	Label-aware cross-product with MPS-accelerated X projections and shared SIMPLS/LDA semantics.
KODAMA evolution	For a fixed classifier: same folds, constrained proposals, classifier-specific score, requested feasible components, and independent M runs.	For a fixed classifier: same folds, constrained proposals, classifier-specific score, requested feasible components, and independent M runs.
Failure semantics	Unavailable CUDA entry points throw; no CPU fallback is labeled CUDA.	Unavailable Metal entry points throw; no CPU fallback is labeled Metal.

3.5 Graph-first input contract

KODAMA.graph returns graph arrays, backend-matched PCA starts, metadata, and timings, but never the source matrix. Raw features remain caller-owned when supplied separately.

Algorithm: `graph preparation` and `KODAMA.matrix dispatch`

`KODAMA.Graph(X, k, backend):`

Table 5: Public API groups in kodama-cpp.

Layer	Public API	Purpose
Cross-validation	KNNCV, PLSLDACV	Fold assignment, prediction, fold accuracies, confusion matrices, timing, memory, and backend metadata.
Core optimization	CoreKNN, CorePLSLDA	Independent label-evolution runs that maximize KNN or PLS-LDA cross-validated accuracy.
Graph preparation	KODAMA.graph / KODAMAGraph	One reusable KNN graph plus backend-matched UMAP/openTSNE PCA starts, without retaining the raw matrix.
KODAMA matrix	KODAMA.matrix / KODAMAMatrix	Landmark selection, splitting, M runs, optimized label vectors, KODAMA dissimilarity, and graph output.
Graph-input KODAMA	KODAMA.matrix.graph / KODAMAMatrixFromGraph	Runs the same KNN or PLS-LDA KODAMA optimizer from supplied neighbor indices/distances; KNN reuses the graph directly and PLS-LDA uses self-tuning Laplacian features when the data matrix is unavailable.
Principal components	PCA / KODAMA.pca	Float32 randomized PCA scores, loadings, singular values, explained variance, preprocessing vectors, backend identity, and runtime on CPU, CUDA, or Metal.
Visualization	KODAMA.visualization	Pinned fastEmbedR UMAP and openTSNE ports from standard or KODAMA-corrected float32 graphs; backend-matched raw-data PCA initialization is the default, fuzzy UMAP weighting is the default, and initialization provenance is returned.
Graph and clustering	makeSNNGraph equivalent and random-walk clustering	CPU, CUDA, or Metal neighbor-graph construction followed by an explicit CPU random-walk clustering kernel.
R compatibility utilities	normalization, scaling, dinisurface, helicoid, spirals, swissroll	Dependency-free R implementations preserve the preprocessing signatures, returned fields, and seeded synthetic-manifold constructions of KODAMA 3.3; they are wrapper conveniences rather than C++ kernels.

```

X32 <- Float32(X)
G <- BackendKNN(X32, k)
(Z_U, Z_T) <- BackendPCAStarts(X32)
return P = {G, Z_U, Z_T, dimensions, backend, timings}
# X32 and X are not retained by P.

```

```

KODAMA.Matrix(input, optional X):
  raw X      -> prepare P; use raw geometry
  prepared P+X -> reuse P; use caller-owned raw geometry
  prepared P  -> reuse P; use self-tuning graph geometry
  bare G      -> use G with no stored PCA starts and graph geometry

```

Table 6: Graph-first KODAMA.matrix input contract.

Input	Working geometry	KNN evolution	PLS-LDA evolution	Graph builds
Raw X	Raw float32 variables	Data-native KNN	Raw variables	One
Prepared P	Self-tuning graph features	Fixed supplied graph	Laplacian surrogate	Zero
Prepared P + X	Raw float32 variables	Fixed supplied graph	Raw variables	Zero
Bare G	Self-tuning graph features	Fixed supplied graph	Laplacian surrogate	Zero

Table 7: MetRef CUDA graph-API smoke validation ($n = 873$, $p = 375$, $k = 100$, $M = T = 10$). Times are core seconds.

Classifier	Input	Builds	Time	Best CV	Agreement
KNN	X	1	0.323	0.954	0.907
KNN	P + X	0	0.037	0.965	0.907
KNN	P	0	0.023	0.983	1.000
KNN	G	0	0.023	0.983	1.000
PLS-LDA	X	1	2.217	0.936	1.000
PLS-LDA	P + X	0	2.088	0.936	1.000
PLS-LDA	P	0	1.817	0.947	1.000
PLS-LDA	G	0	1.801	0.947	1.000

MetRef CUDA smoke validation used $n = 873$, $p = 375$, $k = 100$, and $M = T = 10$. The prepared R object occupied 1.08 MB and contained no raw-data field, compared with 2.70 MB for the input matrix; one-time graph/PCA preparation required 0.342 wall seconds. Exact PLS-LDA agreement confirms that prepared P does not alter raw-feature classification when X is also supplied. The KNN difference is expected: P + X uses fixed-graph voting, whereas X uses data-native KNN within each landmark sample. P and bare G agree exactly because stored PCA starts affect visualization initialization, not graph-only label evolution. These figures validate API lifecycle semantics and are not presented as a performance benchmark.

3.6 Computational scaling

3.7 Graph-input KODAMA validation boundary

Table 8: Dominant computational work in the KODAMA pipeline.

Component	Dominant work	Implementation implication
KODAMA runs	M independent runs, each with Tcycle proposal cycles plus one initial CV pass.	Embarrassingly parallel over M; each cycle performs exactly one CV evaluation.
KNN core	Neighbor search/build plus $O(M \text{ Tcycle } n_L k)$ voting on landmarks, where n_L is the landmark count and k is the KNN predictor size.	Graph-input KNN removes the search/build term and reuses supplied indices and distances.
PLS-LDA core	$O(M \text{ Tcycle folds } C \text{ PLS}(n_{\text{train}}, p, h))$ for h requested components, plus latent-space LDA scoring.	Float32 label-aware SIMPLS and fold buffers reduce constants but do not change the repeated-CV structure.
Graph correction	$O(M E)$ agreement counting for E retained graph edges, followed by row-wise neighbor sorting.	Sparse by default; dense n by n dissimilarities are avoided for scalability.
Visualization	Runs on the base or KODAMA-corrected sparse graph using UMAP/openTSNE helpers.	Visualization time is reported separately because it is not part of the CV optimization objective.

Table 9: Interpretation of graph-input KODAMA paths.

Path	Implementation	Interpretation
KNN graph input	Uses supplied neighbor indices/distances directly in CoreKNNGraph_CPU and KODAMAMatrixFromGraph; no neighbor index is rebuilt.	Preserves graph-based KNN voting mathematics. The precomputed-graph voting kernel is host-resident in the current release; accelerator backends can still execute matrix-side initialization work.
PLS-LDA graph input with X	When the original data matrix is supplied with the graph, PLS-LDA uses float32 X and the graph supplies initialization/projection support.	Equivalent classifier path to data-input PLS-LDA; graph handling is an input-format extension.
PLS-LDA graph-only input	When X is unavailable, a host kernel transforms the graph into self-tuning normalized Laplacian features before backend-selected CorePLSLDA.	Exposed as an optional spectral surrogate, not as evidence that graph-only PLS-LDA matches data-input PLS-LDA in all datasets.
Validation boundary	Wrapper tests exercise both KNN and PLS-LDA graph-input calls; benchmark comparisons must use the same M, Tcycle, seed, and supplied graph.	The manuscript does not use graph-only PLS-LDA results to support data-input KODAMA performance claims.

3.8 KODAMA directly from a KNN matrix

The graph-input API represents a KNN matrix as two rectangular arrays with identical shape n by k . Entry $I[i,j]$ is the sample identifier of the j th neighbor of sample i , and $D[i,j]$ is its non-negative dissimilarity. The boundary accepts either zero-based or one-based identifiers, converts them to the internal convention, discards invalid identifiers, self-neighbors, and non-finite distances, clamps negative distances to zero, and retains at most `graph.neighbors` entries per row. The row order must match the sample order used by every optional label, constraint, and feature vector.

For graph-only KNN KODAMA, the supplied graph replaces construction of the global neighbor index and remains the source of neighbors for CV voting, landmark-label projection, and final agreement correction. A deterministic self-tuning normalized-Laplacian representation is still computed once to provide geometry for landmark selection and initial splitting; it does not replace graph-based KNN voting. If the original data matrix X is available, `KODAMAMatrixFromGraphData` uses float32 X for that geometry while continuing to reuse the supplied graph.

PLS-LDA requires a rectangular feature representation. With X supplied, graph-input PLS-LDA uses the ordinary float32 SIMPLS plus latent-space LDA classifier on X . With only I and D supplied, it uses the deterministic self-tuning normalized-Laplacian features described above. This graph-only PLS-LDA route is an explicit spectral surrogate and is not claimed to be numerically equivalent to data-input PLS-LDA. Graph normalization and spectral feature extraction are host operations in the current release; CPU, CUDA, or Metal is then selected for the PLS-LDA evolution.

The remaining KODAMA procedure is unchanged: each of M independent runs selects landmarks, initializes at most splitting classes, performs Tcycle proposal and CV updates, and projects its optimized labels to all samples. The output contains the M by n label matrix, one best raw CV accuracy per run, one graph in either base or KODAMA-corrected state, timing fields including graph-feature time, and backend metadata. UMAP or openTSNE can then consume the corrected graph directly; when X is unavailable, visualization reports its graph-only initialization fallback.

The R and Python graph-input functions expose the same contract. Python uses the direct snake_case transliteration of dotted R names. Defaults, accepted values, the one-based best-run convention, and returned fields are otherwise aligned.

```

C++17
kodama::NeighborGraph g{I, D, k};
kodama::KODAMAMatrixOptions opt;
opt.runs = 100; opt.cycles = 100; opt.classifier = kodama::CoreClassifier::KNN;
auto fit = kodama::KODAMAMatrixFromGraph(g, n, {}, {}, {}, opt);

R
p <- KODAMA.graph(X, k = 100, backend = "cuda")
fit_graph <- KODAMA.matrix(graph = p, classifier = "knn")
fit_graph_x <- KODAMA.matrix(data = X, graph = p, classifier = "pls_lda")
fit_raw <- KODAMA.matrix(data = X, classifier = "pls_lda")
fit_bare <- KODAMA.matrix(graph = list(indices = I, distances = D), classifier = "knn")

Python
p = kodama.graph(X, k=100, backend="cuda")
fit_graph = kodama.matrix(graph=p, classifier="knn")
fit_graph_x = kodama.matrix(data=X, graph=p, classifier="pls_lda")
fit_raw = kodama.matrix(data=X, classifier="pls_lda")
fit_bare = kodama.matrix(graph={"indices": I, "distances": D}, classifier="knn")

```

4 Relationship to public KODAMA literature

The contribution of kodama-cpp is not a new definition of KODAMA. It is a reorganization of the published method into a reusable numerical system. The 2014 PNAS paper is the methodological reference for accuracy maximization. The 2017 Bioinformatics paper is the software reference for the R package, including KNN and PLS-DA classifiers, repeated suboptimal runs, and construction of a dissimilarity matrix from the optimized label vectors. The R documentation is the public reference for defaults such as $M = 100$, $T_{\text{cycle}} = 20$, 75% sample selection, and the output fields dissimilarity, accuracy, proximity, classification vectors, scores, entropy, and landmarks.

The main differences are architectural and computational. The R package owns a complete R workflow and uses R-package dependencies for data handling, nearest neighbors, and visualization. kodama-cpp owns the numerical kernels and exposes them through C++ types so that R and Python wrappers can share one backend. This makes the comparison with the previous version a compatibility question: held-out label predictability remains the central signal and the output roles can be mapped, while exact trajectories need not match because the current standard adds grouped adaptive proposals, transition-driven coarsening, and a disclosed degeneracy guard. Float32 storage, package-owned CPU/CUDA/Metal search, accelerator kernels, and wrapper-independent graph objects are additional implementation changes.

4.1 Implementation novelty relative to the R package

4.2 Relationship to semi-supervised learning and cluster validation

KODAMA is related to, but distinct from, semi-supervised learning and weak supervision. Classical semi-supervised graph methods assume that at least some labels are observed and then propagate them under smoothness assumptions, as in Gaussian-field label propagation and local-global consistency. Pseudo-labeling and weak-supervision systems also create or aggregate imperfect labels, but they typically use those labels to train a downstream predictor. KODAMA reverses the emphasis: the label vector itself is the optimization

Table 10: Compatibility with the public KODAMA R interface and literature.

Aspect	Public R/literature contract	kodama-cpp
Objective	KODAMA maximizes cross-validated accuracy of an evolving label vector.	Held-out label predictability remains the classifier signal and raw accuracy is reported; the standard search ranks proposals with a disclosed label-only degeneracy guard.
Search dynamics	The historical core evolves labels through its published stochastic accuracy-maximization procedure.	Grouped adaptive moves and transition-driven coarsening are explicit search extensions; they use CV predictions but never reference labels.
Independent runs	R documentation reports M iterative processes and Tcycle optimization cycles.	KODAMAMatrixOptions exposes runs and cycles; runs remain independent across CPU, CUDA, and Metal backends.
Sample selection	The documented FUN_SAM default selects 75% of samples per iterative process.	Landmark and splitting controls preserve the same sampling role while exposing it in typed options.
Classifiers	The R package documents KNN and PLS-DA classifiers.	The KODAMA optimizer exposes KNN and SIMPLS + PLS-LDA; component count is explicit.
Output contract	The R interface returns dissimilarity, accuracy, proximity, label vectors, scores, entropy, and landmarks.	The C++ result returns optimized labels, accuracies, graphs, timings, memory, and backend metadata for wrappers.
Architecture	The R package is built around R workflows and R dependencies.	The core is R/Python independent, CMake-installable, and accelerated where available.

object, and its quality is the ability of a chosen classifier to reproduce it under held-out folds.

The closest evaluation literature is cluster validation by stability or prediction. Methods such as stability-based clustering and prediction strength ask whether a grouping is reproducible under perturbation or held-out prediction. KODAMA uses the same general principle of reproducibility, but internalizes it as the search objective over label vectors. This is why the benchmark reports the internal CV score together with adjusted Rand index, silhouette, local purity, and active class count rather than presenting CV accuracy alone as proof of external correctness.

The manuscript also follows warnings from the model-selection and circular-analysis literature. Cross-validation can be biased when the same data are used both to choose a model and to estimate final performance; feature selection outside the resampling loop is a classic example. In kodama-cpp, the internal CV score is intentionally the search criterion, so external claims must come from separate diagnostics, transparent parameter sensitivity, and comparisons with standard UMAP and t-SNE on the same datasets.

Table 11: Implementation innovations in kodama-cpp.

Innovation	What changes in kodama-cpp
State ownership	Moves folds, labels, classifier workspaces, and graph outputs from repeated R-level calls into reusable C++ objects.
Backend-native classifiers	Keeps each classifier’s mathematics consistent across CPU, CUDA, and Metal while implementing its dominant operations separately for each backend.
Float32 execution	Uses float32 analysis matrices and accelerator workspaces end to end while accepting wrapper-provided double inputs at the boundary.
Label-aware SIMPLS	Computes PLS-LDA response cross-products from compact labels instead of dense one-hot matrices and evaluates the requested feasible component count.
Accelerated search	Provides package-owned CPU HNSW, CUDA exact/recall-tuned IVF-Flat search, and Metal exact/recall-tuned IVF-Flat search.
Reproducible heterogeneity	Keeps M runs independent and reports actual backend and search metadata so acceleration can be tested without altering the objective.
Search evolution	Makes grouped adaptive proposals, transition-driven class moves, and the label-only degeneracy guard explicit and separately reports their raw CV accuracy and acceptance score.

Table 12: Literature positioning of KODAMA.

Literature area	Typical goal	Role of KODAMA
Semi-supervised graph learning	Label propagation and local-global consistency start from observed labels and smooth them on a graph.	KODAMA starts from synthetic or supplied candidate labels and optimizes their cross-validated predictability.
Pseudo-labeling and weak supervision	Pseudo-labeling and data programming create imperfect labels to train predictive models.	KODAMA treats the label vector as the object being discovered; the classifier is the measuring instrument.
Cluster stability and prediction strength	Stability methods evaluate whether clusters survive perturbation or prediction on held-out samples.	KODAMA turns held-out predictability into an optimization objective, then reports external diagnostics separately.
Manifold visualization	UMAP and t-SNE optimize low-dimensional layouts from neighbor or affinity structures.	KODAMA supplies a corrected graph to the same visualization step; it does not replace those embedding objectives.
Open-source ML software	JMLR software papers emphasize clear APIs, reproducibility, tests, licensing, and benchmark transparency.	kodama-cpp separates the numerical core from R/Python wrappers and records backend parameters and validation evidence.

5 Evaluation

We evaluate kodama-cpp at three levels. First, kernel benchmarks isolate KNNCV and PLSLDACV from the full KODAMA matrix pipeline so that nearest-neighbor search and

PLS-LDA fitting can be interpreted separately. Second, core optimizer benchmarks measure repeated label-vector evolution over multiple seeds. Third, matrix-level validation measures end-to-end KODAMA.matrix calls and reports both cross-validated accuracy and external-label diagnostics where reference labels are available.

The evaluation is intentionally separated into these layers because KODAMA has several scaling regimes. KNN acceleration is dominated by neighbor search and voting. PLS-LDA acceleration is dominated by repeated SIMPLS and LDA work across folds and cycles. The final graph and visualization stages depend primarily on the sparse graph size. Reporting the layers separately makes it possible to attribute a speedup to the part of the implementation that produced it.

The evaluation also separates optimization from validation. Held-out CV accuracy is the internal classifier signal, while the current standard ranks proposals with the disclosed label-only acceptance score. Neither quantity is interpreted as external biological, chemical, or semantic truth. When reference labels are available, they are held outside the optimizer and reported after the fact. This distinction follows the broader cautionary literature on model selection, feature selection bias, and circular analysis.

All timings in this section are wall-clock seconds from the benchmark drivers, and all accuracy values are computed on held-out cross-validation folds. CPU kernel measurements use the configured single-thread CPU path unless explicitly labeled otherwise; CUDA measurements use the same CUDA Toolkit runtime used by the test suite. Tables are intentionally separated by scope: kernel-level rows do not claim end-to-end speedups, and graph-input rows are treated as API validation unless paired with data-input KODAMA results under the same M, Tcycle, seed, and graph settings.

5.1 Benchmark protocol and data coverage

5.2 Evaluation guardrails and ablations

5.3 Dependency-free CPU and Apple Metal validation

The dependency-light backend experiment was run on macOS with CUDA and OpenMP disabled. The CPU build therefore measures the package-owned HNSW and float32 SIMPLS/LDA implementations, while the Metal build links only Apple system frameworks. CPU/Metal agreement within the documented float32 tolerance is the primary acceptance criterion; timings are secondary evidence.

A focused Apple M3 graph benchmark used 4,000 samples, 32 float32 variables, $k = 30$, and three repeated runs. Before concurrent insertion, the accepted implementation builds a deterministic connected seed containing $\min(n, \max(n_threads + 1, 2 \cdot m_HNSW + 1))$ points, where m_HNSW is the internal graph connectivity. Parallel HNSW construction required a median 6.722 s with one thread and 1.791 s with four threads, a 3.75-fold scaling gain. The earlier four-thread serial-insertion implementation required 7.248 s, so the accepted implementation was 4.05-fold faster. Four-thread overlap with the one-thread graph had a three-run median of 0.99966. Across 100 repeated four-thread builds of a 600 by 24 test problem with $k = 20$, minimum and mean brute-force recall were 0.99975 and 0.99987. Both serial and parallel paths must retain at least 0.99 exact-neighbor recall in CTest, and the complete CPU suite passes ThreadSanitizer.

Table 13: Benchmark protocol for the current evaluation.

Item	Value
Date	2026-07-27 UTC for the matched current-commit MetRef CPU/CUDA run; earlier validation snapshots are dated explicitly
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB device memory, driver 595.71.05
Build validation	Fresh CUDA 13.0 compute-capability-12.0 build at 9da48ee succeeded; CTest passed 3/3 and the installed R wrapper passed test-that
Runtime	CUDA Toolkit only for native search/PLS paths; no FAISS, cuVS, RAFT, RMM, or Armadillo link
Data format	RData lists exported to contiguous float32 row-major matrices
CPU setting	Four CPU threads for the matched MetRef end-to-end run; other tables state their worker count
CV kernels	KNNCV and PLSLDACV measured independently from the full matrix pipeline
Core optimizer	Three seeds per dataset for CoreKNN and CorePLSLDA medians
Reported metrics	Wall time, peak memory where available, CV accuracy, ARI, and active class count

Table 14: Datasets copied into the benchmark data root for the current evaluation.

Dataset	Samples	Variables	Classes
COIL20	1,440	16,384	20
FashionMNIST	70,000	784	10
FlowRepository	5,220,347	32	13
MNIST	70,000	784	10
Macosko2015_retina	44,808	50	12
MetRef	873	375	22
TabulaMuris	70,118	50	56
USPS	11,000	256	10
flow18	1,000,021	11	16
imagenet	1,281,167	1,024	1,000
mass41	965,282	14	17

Metal IVF-Flat remains an explicit option rather than an automatic replacement for exact search. On MNIST10k, exact and recall-tuned IVF KNN CV both produced accuracy 0.9490, while elapsed time changed from 2.054 to 1.662 s. A 0.99 pilot-recall target was rejected because accuracy decreased to 0.9482; the accepted automatic pilot target is 0.999. End-to-end KODAMA can still favor exact search when repeated IVF training costs more than the saved query time.

Table 15: Evaluation safeguards used to avoid over-interpreting the internal CV objective.

Concern	Manuscript position	Evidence reported or controlled
Circularity	CV accuracy is the internal objective, not an external truth score.	Report ARI, silhouette, local purity, active classes, and standard embedding baselines after optimization.
Overclaiming	Present kodama-cpp as a KODAMA implementation and acceleration library, not a new universal learning paradigm.	State methodological continuity with the 2014 and 2017 KODAMA publications and limit novelty claims to architecture and numerics.
Parameter dependence	M, Tcycle, landmarks, splitting, KNN k, graph k, and PLS components can affect results.	Use predefined sensitivity experiments and report parameter settings in every benchmark table and plot.
Visualization bias	A visually appealing layout is not itself evidence of label quality.	Run classic UMAP/openTSNE and KODAMA-corrected UMAP/openTSNE with the same implementation and record compactness metrics.
Runtime attribution	End-to-end speedups can hide which kernel produced the benefit.	Separate CV kernels, core optimizer, KODAMA.matrix, graph construction, and visualization timings.
Reproducibility	Wrapper behavior must match the standalone core.	Record CMake settings, backend metadata, R CMD check, pytest, CTest, seeds, and benchmark command lines.

5.4 CUDA workstation validation

The benchmark corpus uses copied float32 RData datasets spanning $n = 873$ to 5,220,347 samples and $p = 11$ to 16,384 variables. For the current matched MetRef result, commit 9da48ee was rebuilt with CUDA 13.0 for compute capability 12.0; all three configured CTest targets and the installed R-wrapper testthat suite passed. Broader tables retain explicitly dated release-history snapshots rather than silently mixing them with this current-commit run.

The current package-owned CUDA IVF-Flat path was checked on MNIST70k and MetRef with five-fold cosine KNN CV and $k = 10$. On MNIST70k it produced accuracy 0.973857 in 4.233 s with $nlist = 237$ and maximum $nprobe = 32$, compared with 0.973029 in 4.753 s for the recorded former FAISS/cuVS row. On MetRef it produced 0.816724 in 0.195 s, compared with 0.814433 in 0.297 s. These are implementation spot checks rather than repeated-run performance estimates.

A focused resident-IVF microbenchmark separated construction from repeated search on a deterministic 50,000 by 32 float32 matrix using five timed searches after one warm-up. For all-row self-search, retaining the index reduced mean rebuild-plus-search time from 0.8695 to 0.8091 s on Metal and from 0.1984 to 0.1765 s on CUDA (1.07x and 1.12x). For 1,000-query batches against the same training matrix, time changed from 0.1179 to 0.0306 s on Metal and from 0.0318 to 0.0092 s on CUDA (3.86x and 3.45x). Neighbor overlap was

Table 16: Ablation matrix for the release benchmark.

Ablation	Comparison	Question answered
Search evolution	Raw-accuracy proposals versus the standard grouped/guarded search	Separates the current search extension from backend acceleration and from the unchanged CV classifiers.
Classifier	KNN versus PLS-LDA	Tests whether local-neighbor and latent-linear predictability give complementary label vectors.
Graph correction	Standard graph versus KODAMA-corrected graph	Isolates whether the optimized label ensemble improves downstream UMAP/openTSNE structure.
Search depth	Tcycle = 20, 50, 100	Measures whether additional proposal cycles improve CV accuracy or reduce fragmentation.
Ensemble size	M = 20, 50, 100	Measures whether more independent runs stabilize the final graph and best-run diagnostics.
Landmark and splitting controls	Predeclared values, not per-dataset visual tuning	Checks sensitivity of the initial label space and directly optimized sample subset.
Backend	Single-core CPU, 4-core MultiCPU, CUDA, and Apple Metal where available	Separates mathematical equivalence from scheduling and hardware acceleration.
Wrapper parity	C++ core, R wrapper, Python wrapper	Ensures language bindings are thin and do not introduce independent numerical behavior.

Table 17: Dependency-free CPU and native Apple Metal validation. Quality entries are CPU/Metal where two values are shown.

Dataset	Scope	CPU s	Metal s	Speedup	Quality
MetRef	KNNCV	11.145	0.026	425x	accuracy 0.827033 / 0.827033
MetRef	PLSLDACV, 50 components, five-run median	0.790	0.202	3.90x	accuracy 0.990836 / 0.989691
MetRef	KODAMA KNN, M=10 T=20	21.709	0.171	127x	CV 0.978626; ARI 0.093922; 15 classes
MetRef	KODAMA PLS-LDA, M=8 T=20	14.930	6.900	2.16x	best CV 0.963359 / 0.969466; ARI 0.741026 / 0.849732

1.000 in every comparison. These are systems measurements, not dataset-level accuracy estimates.

The current $M = 100$, $T_{\text{cycle}} = 100$ MetRef validation shows why cross-validated accuracy is reported together with label-quality diagnostics. Matched CPU and CUDA PLS-LDA runs both reached maximum CV accuracy 1.000 and median CV accuracy 0.992366. Their CV-selected runs contained 24/27 classes and attained ARI 0.7654/0.7732 against 22 withheld reference classes. Backend-native $k = 30$ fuzzy UMAP produced truth-label silhouette 0.8206/0.7856. Because finite-precision and scheduling differences can select different stochastic runs, these results support matched objective behavior rather than bitwise trajectory identity.

The dependency-distilled CUDA build links CUDA Toolkit libraries but no FAISS, cuVS, RAFT, or RMM soname. Binary wrappers therefore need to locate the CUDA runtime selected at build time, but do not need to provision the removed search libraries.

The exact-quota selector was also isolated on flow18 (1,000,021 rows, 11 variables). With a request exceeding n , the historical rule yielded 750,016 effective landmarks. Matrix-only sampling from 300 coarse strata required 0.321 seconds and represented all 300 strata. A CUDA KNN run with $M = 1$ and $T_{\text{cycle}} = 10$ completed in 362.348 seconds and reached raw CV accuracy 0.963941; the shared 100-neighbor graph required 335.079 seconds, whereas the reported optimization sum was 1.734 seconds. The experiment therefore attributes the remaining large-data cost to global graph construction rather than landmark selection.

Table 18: Dependency-distilled native CUDA KNN spot checks.

Dataset	Path	Accuracy	Seconds	nlist	max nprobe	Pilot recall
MNIST70k	native IVF-Flat	0.973857	4.233	237	32	0.994531
MetRef	native IVF-Flat	0.816724	0.195	27	20	0.991406

5.4.1 EARLIER ACCELERATOR VALIDATION SNAPSHOT

Table 19: Earlier CPU/CUDA cross-validation kernel measurements retained for release-history continuity. Accuracy is reported as CPU/CUDA.

Dataset	Kernel	CPU s	CUDA s	Speedup	Acc CPU/CUDA	Delta	Comp.
COIL20	KNN CV	5.455	1.435	3.8x	0.917/0.916	-0.000695	-
MNIST	KNN CV	76.769	4.753	16.2x	0.974/0.973	-0.000928	-
MetRef	KNN CV	0.163	0.297	0.6x	0.821/0.814	-0.006873	-
MetRef	PLSLDACV	3.196	0.118	27.2x	0.992/0.992	0.000000	50
USPS	KNN CV	2.273	0.429	5.3x	0.688/0.688	-0.000819	-
USPS	PLSLDACV	3.760	0.158	23.8x	0.684/0.687	+0.002727	50
mass41	KNN CV	366.883	3.692	99.4x	0.912/0.912	-0.000066	-

5.4.2 CORE OPTIMIZER MEDIANS

Table 20: Medians for the CoreKNN and CorePLSLDA optimization paths. Accuracy and ARI are reported as CPU/CUDA.

Dataset	Core	CPU s	CUDA s	Speedup	Acc	ARI	Classes
MetRef	CoreKNN	0.147	0.122	1.2x	0.963/0.947	0.271/0.313	16/16
MetRef	CorePLSLDA	16.597	0.626	26.5x	0.885/0.838	0.357/0.366	53/48
USPS	CoreKNN	2.282	0.242	9.4x	0.969/0.968	0.216/0.240	59/53
USPS	CorePLSLDA	20.189	1.950	10.4x	0.938/0.910	0.140/0.146	80/77

5.4.3 MULTICPU VERSUS GPU KODAMA COMPARISON

The backend comparison is separated by scope. Core-optimizer rows measure the repeated CV label-search kernels without final graph construction, while the KODAMA.matrix row includes landmark selection, label search, projection, and graph correction on a small end-to-end validation dataset. This prevents kernel timings from being reported as full pipeline timings.

The rows below therefore state their scope explicitly. CPU rows use the configured CPU backend, CUDA rows use the CUDA backend, and differences in CPU/CUDA accuracy are treated as reproducibility tolerances caused by stochastic search, float32 arithmetic, and approximate nearest-neighbor behavior rather than as evidence of different mathematical objectives.

Table 21: KODAMA core optimizer CPU/CUDA comparison. Scope is reported explicitly because these rows do not include final graph construction.

Dataset	Classifier	Backend	Seconds	CV acc	ARI	Classes	Scope
MetRef	KNN	CPU	0.147	0.963	0.271	16	core optimizer
MetRef	KNN	CUDA	0.122	0.947	0.313	16	core optimizer
MetRef	PLS-LDA	CPU	16.597	0.885	0.357	53	core optimizer
MetRef	PLS-LDA	CUDA	0.626	0.838	0.366	48	core optimizer
USPS	KNN	CPU	2.282	0.969	0.216	59	core optimizer
USPS	KNN	CUDA	0.242	0.968	0.240	53	core optimizer
USPS	PLS-LDA	CPU	20.189	0.938	0.140	80	core optimizer
USPS	PLS-LDA	CUDA	1.950	0.910	0.146	77	core optimizer

5.4.4 LEGACY KNN-COMPATIBLE PREDECESSOR COMPARISON

A legacy KNN-compatible predecessor experiment used MetRef, KNN, $M = 100$, $T_{\text{cycle}} = 100$, 655 effective landmarks under the historical 75% rule, splitting = 50, $k = 30$, graph $k = 100$, and seed 1234. Splitting = 50 matches the internal k-means initialization in KODAMA R 2.4.1. This legacy release is used because it exposes the selectable KNN path needed for a KNN-to-KNN comparison; it is not presented as the current CRAN release. PLS is excluded because the historical PLS-DA decoder and current SIMPLS plus latent-space LDA are different classifiers.

The current grouped and guarded proposals extend the historical stochastic search, so this is a matched-settings predecessor and systems comparison rather than label-trajectory parity. Current single-core execution was 1.58 times slower than KODAMA R 2.4.1 on this small case, whereas four-core execution was 2.59 times faster and CUDA was 258.6 times faster. Every row reached best raw CV accuracy 1.000; the different median scores and label diagnostics are reported rather than interpreted as backend equivalence.

Table 22: Legacy KNN-compatible MetRef comparison at $M = 100$ and $T_{\text{cycle}} = 100$. Speedup is relative to KODAMA R 2.4.1; this is not the current CRAN release.

Implementation	Backend	Workers	Seconds	Speedup	CV best/median	ARI selected/median	Classes selected/median
KODAMA R 2.4.1	CPU	1	610.543	1.00x	1.000/0.961	0.000/0.054	2/6
kodama-cpp	CPU	1	965.348	0.63x	1.000/0.874	0.045/0.071	2/8.5
kodama-cpp	CPU	4	235.547	2.59x	1.000/0.874	0.045/0.071	2/8.5
kodama-cpp	CUDA	4	2.361	258.6x	1.000/0.882	0.052/0.069	3/8.5

5.4.5 CURRENT CRAN SYSTEMS COMPARISON

A separate short-run systems comparison addresses the current public release. MetRef was run with $M = 20$, $T_{\text{cycle}} = 20$, requested landmarks = 100000 (655 effective landmarks under the retained 75% rule), splitting = 100, graph $k = 500$, $n_{\text{comp}} = 50$, seed 1234, and four CPU workers. The CV-selected run is chosen by raw held-out accuracy; ARI and class count are computed only afterward. KODAMA 3.3 automatically selects its PLS route because its deprecated FUN argument is ignored, so this is not classifier- or trajectory-parity with kodama-cpp SIMPLS plus latent-space LDA.

Current CRAN KODAMA 3.3 completed in 344.914 seconds, compared with 822.989 seconds for kodama-cpp CPU4; thus KODAMA 3.3 was 2.39 times faster in this limited check. It also had higher selected raw CV accuracy and selected ARI. These unfavorable current results are retained and are not combined with the classifier-matched legacy KNN comparison. An exactly matched CUDA row is omitted because graph $k = 500$ exceeds the native CUDA builder limit of 256; neither landmarks nor graph k was changed silently to create an accelerator row.

Table 23: Short MetRef current-release systems comparison at $M = 20$ and $T_{\text{cycle}} = 20$. KODAMA 3.3 and kodama-cpp use different PLS classifier routes; this is not classifier parity.

Implementation	Classifier route	Backend	Workers	Seconds	CV selected/median	ARI selected/median	Classes selected/median
KODAMA 3.3	automatic PLS	CPU	4	344.914	0.9893 / 0.9740	0.7004 / 0.6961	20 / 20
kodama-cpp 0.1.0	SIMPLS + LDA	CPU	4	822.989	0.9802 / 0.9550	0.6430 / 0.6584	30 / 35

Table 24: Matched current-commit KODAMA.matrix PLS-LDA validation on MetRef with $M = 100$ and $T_{\text{cycle}} = 100$. ARI and silhouette use external labels only after optimization.

Backend/classifier	Wall s	Core s	Max/median CV	Selected ARI/truth sil.	Selected classes	Peak MB
CPU PLS-LDA	341.648	341.612	1.0000/0.9924	0.7654/0.8206	24	351.4
CUDA PLS-LDA	270.408	270.220	1.0000/0.9924	0.7732/0.7856	27	729.7



Figure 2: Historical 2026-07-20 MetRef embedding-parity diagnostic after the float32 PLS-LDA optimization. Classic data, CPU KODAMA, and CUDA KODAMA were deliberately rendered through the same CPU embedding implementation and PCA initialization to isolate the KODAMA backend. This is not the current operational backend-native visualization contract. Colors are external reference labels withheld from optimization.

5.4.6 CLASSIC VERSUS KODAMA VISUALIZATION VALIDATION

Because the final KODAMA object is often interpreted through a two-dimensional layout, we added an explicit visualization comparison against standard UMAP and openTSNE. The driver runs the same embedding implementation on either the ordinary neighbor graph or the KODAMA-corrected graph, then reports truth-label silhouette, local label purity, KODAMA label ARI, active classes, and wall-clock time. This makes the visualization comparison separate from the CV objective and prevents the evaluation from relying on a single favorable small dataset.

The release-validation panel used CUDA with $M = 100$, $T_{\text{cycle}} = 100$, landmarks = 100000 subject to the historical 75% rule, KNN predictor $k = 30$, graph $k = 100$, embedding $k = 30$, openTSNE perplexity = 30, and seed 1234. It evaluated MetRef, PBMC3K PCA50, OptDigits, USPS, and Macosko2015 retina with both KNN and PLS-LDA. Reference labels were withheld from optimization and used only for the reported ARI, silhouette, purity, and compactness diagnostics.

The result is deliberately mixed. PLS-LDA increased truth-label silhouette on both embeddings for MetRef (the current matched UMAP check changed 0.119 to 0.844; the earlier openTSNE check changed 0.067 to 0.530) and PBMC3K PCA50 (0.410 to 0.484; 0.349 to 0.402), but reduced it on OptDigits, USPS, and Macosko2015 retina. KNN reduced silhouette on all five datasets despite best CV accuracy between 0.9927 and 1.0000. Thus a highly predictable optimized label vector is not sufficient evidence of improved external-label recovery or visualization, and no universal visualization-improvement claim is made.

Complete KODAMA.matrix wall time ranged from 2.967 to 97.295 seconds for KNN and from 486.707 to 708.579 seconds for PLS-LDA. The paired embeddings themselves

required less than 0.88 seconds each in this panel, so the reported timing distinction correctly attributes the dominant cost to label optimization rather than to UMAP or openTSNE.

Table 25: CUDA M=100, Tcycle=100 comparison of standard UMAP/openTSNE with KODAMA-corrected graphs. Silhouette and purity are computed against external labels used only after optimization; ARI and class count refer to the CV-selected KODAMA label vector.

Dataset	Classifier	Embedding	Sil. classic $\rightarrow$ KODAMA	Δ sil.	Purity classic $\rightarrow$ KODAMA	ARI/classes	KODAMA s
MetRef	KNN	UMAP	0.105 $\rightarrow$ 0.016	-0.090	0.590 $\rightarrow$ 0.575	0.049 / 3	3.381
MetRef	KNN	openTSNE	0.067 $\rightarrow$ 0.018	-0.049	0.616 $\rightarrow$ 0.608	0.049 / 3	3.381
MetRef	PLS-LDA	UMAP	0.119 $\rightarrow$ 0.844	+0.725	not recomputed	0.832 / 21	173.104
MetRef	PLS-LDA	openTSNE	0.067 $\rightarrow$ 0.530	+0.463	0.616 $\rightarrow$ 0.904	0.879 / 28	589.633
PBMC3K PCA50	KNN	UMAP	0.410 $\rightarrow$ 0.387	-0.023	0.830 $\rightarrow$ 0.796	0.374 / 3	2.967
PBMC3K PCA50	KNN	openTSNE	0.349 $\rightarrow$ 0.336	-0.013	0.826 $\rightarrow$ 0.805	0.374 / 3	2.967
PBMC3K PCA50	PLS-LDA	UMAP	0.410 $\rightarrow$ 0.484	+0.075	0.830 $\rightarrow$ 0.820	0.594 / 11	486.707
PBMC3K PCA50	PLS-LDA	openTSNE	0.349 $\rightarrow$ 0.402	+0.053	0.826 $\rightarrow$ 0.825	0.594 / 11	486.707
OptDigits	KNN	UMAP	0.700 $\rightarrow$ 0.600	-0.100	0.974 $\rightarrow$ 0.949	0.603 / 28	3.920
OptDigits	KNN	openTSNE	0.563 $\rightarrow$ 0.446	-0.117	0.972 $\rightarrow$ 0.959	0.603 / 28	3.920
OptDigits	PLS-LDA	UMAP	0.700 $\rightarrow$ 0.548	-0.153	0.974 $\rightarrow$ 0.944	0.350 / 47	496.673
OptDigits	PLS-LDA	openTSNE	0.563 $\rightarrow$ 0.470	-0.094	0.972 $\rightarrow$ 0.960	0.350 / 47	496.673
USPS	KNN	UMAP	0.153 $\rightarrow$ 0.071	-0.081	0.698 $\rightarrow$ 0.651	0.300 / 34	19.141
USPS	KNN	openTSNE	0.212 $\rightarrow$ 0.090	-0.121	0.711 $\rightarrow$ 0.672	0.300 / 34	19.141
USPS	PLS-LDA	UMAP	0.153 $\rightarrow$ 0.082	-0.071	0.698 $\rightarrow$ 0.598	0.109 / 70	663.782
USPS	PLS-LDA	openTSNE	0.212 $\rightarrow$ 0.081	-0.131	0.711 $\rightarrow$ 0.643	0.109 / 70	663.782
Macosko retina	KNN	UMAP	0.469 $\rightarrow$ 0.183	-0.286	0.965 $\rightarrow$ 0.953	0.171 / 73	97.295
Macosko retina	KNN	openTSNE	0.171 $\rightarrow$ -0.022	-0.193	0.962 $\rightarrow$ 0.959	0.171 / 73	97.295
Macosko retina	PLS-LDA	UMAP	0.469 $\rightarrow$ 0.176	-0.293	0.965 $\rightarrow$ 0.953	0.410 / 30	708.579
Macosko retina	PLS-LDA	openTSNE	0.171 $\rightarrow$ 0.060	-0.111	0.962 $\rightarrow$ 0.958	0.410 / 30	708.579

5.5 Sensitivity of M and Tcycle

Because KODAMA has two nested stochastic controls, M and Tcycle cannot be justified by visual quality alone. Tcycle controls the depth of label-vector evolution inside one run, whereas M controls how many independent local optima enter the final ensemble. The named release-validation experiment uses MetRef and USPS with both KODAMA classifiers: fixing M = 100 while varying Tcycle in {20, 50, 100}, and fixing Tcycle = 100 while varying M in {20, 50, 100}. It uses landmarks = 100000 subject to the historical 75% rule when n is no larger than landmarks, the default splitting rule, ncomp = 50, knn.k = 30, and a recorded seed.

The Tcycle sweep reports best and median cross-validated accuracy, adjusted Rand index against reference labels used only after optimization, active-class count, and run-time. On USPS, increasing Tcycle from 20 to 100 raised best/median CV accuracy from 0.9662/0.9414 to 0.9997/0.9925 for KNN and from 0.9383/0.9220 to 0.9612/0.9520 for PLS-LDA; median class counts fell from 61 to 32 and from 88.5 to 71. On MetRef, PLS-LDA likewise improved its median CV accuracy from 0.9504 to 0.9924 and reduced median classes from 34 to 24, whereas KNN changed little and remained over-coarsened. Tcycle = 100 is therefore retained as a conservative search-depth setting, not as a claim of universal optimality; the MetRef PLS-LDA selected-run ARI was highest at Tcycle = 50.

The M sweep has a different interpretation. Increasing M raises the number of independent solutions entering the final agreement graph; it is not a search-depth parameter and is not expected to improve the externally scored best run monotonically. Each edge weight

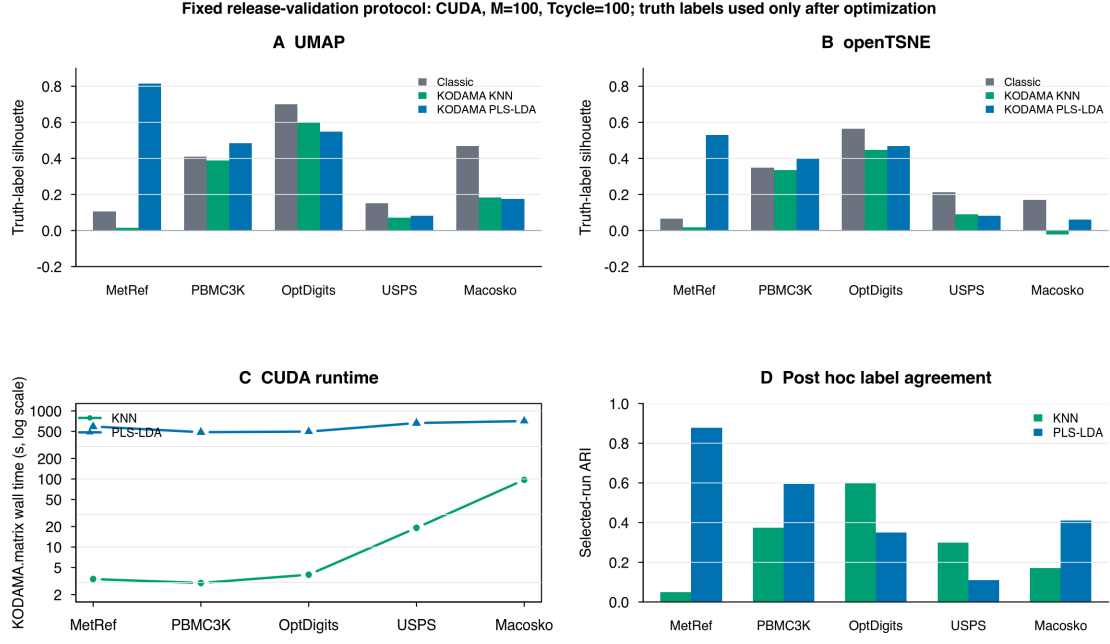


Figure 3: Fixed $M = 100$ and $T_{\text{cycle}} = 100$ matrix-input visualization validation. Panels A and B compare truth-label silhouette; panel C reports complete KODAMA.matrix wall time on a logarithmic scale; panel D reports selected-run ARI. Reference labels were used only after optimization.

is the fraction of runs assigning its two samples to the same label. Its Bernoulli worst-case Monte Carlo standard error is $1/(2 \sqrt{M})$: 0.1118 at $M = 20$, 0.0707 at $M = 50$, and 0.0500 at $M = 100$. Empirically, the graph-weight RMSE relative to the $M = 100$ ensemble fell between $M = 20$ and $M = 50$ for every dataset/classifier pair, reaching 0.0180-0.0407 at $M = 50$ with correlations of 0.9888-0.9988. $M = 100$ is retained to bound ensemble uncertainty, while the nonmonotone selected-run ARI is reported rather than used to tune M .

Table 26: Tcycle sensitivity at fixed $M = 100$. Accuracy entries show $T_{\text{cycle}} = 20$ to $T_{\text{cycle}} = 100$.

Dataset	Classifier	Best acc	Median acc	Median classes	Seconds
MetRef	KNN	0.9924 -> 1.0000	0.8847 -> 0.8809	7 -> 9	3.116 -> 3.115
MetRef	PLS-LDA	0.9786 -> 1.0000	0.9504 -> 0.9924	34 -> 24	119.790 -> 584.089
USPS	KNN	0.9662 -> 0.9997	0.9414 -> 0.9925	61 -> 32	15.336 -> 15.828
USPS	PLS-LDA	0.9383 -> 0.9612	0.9220 -> 0.9520	88.5 -> 71	139.135 -> 636.650

Table 27: M sensitivity at fixed Tcycle = 100. Entries show M = 20 / 50 / 100.

Dataset	Classifier	Best acc	Median acc	Selected ARI	Seconds
MetRef	KNN	1.0000 / 1.0000 / 1.0000	0.9229 / 0.8931 / 0.8809	0.0517 / 0.0517 / 0.0517	0.464 / 1.235 / 3.115
MetRef	PLS-LDA	0.9985 / 1.0000 / 1.0000	0.9901 / 0.9939 / 0.9924	0.8257 / 0.8439 / 0.7700	113.381 / 277.456 / 584.089
USPS	KNN	0.9976 / 0.9976 / 0.9997	0.9947 / 0.9925 / 0.9925	0.3370 / 0.3370 / 0.3139	3.431 / 8.227 / 15.828
USPS	PLS-LDA	0.9639 / 0.9602 / 0.9612	0.9534 / 0.9503 / 0.9520	0.1098 / 0.1080 / 0.1156	123.851 / 319.838 / 636.650

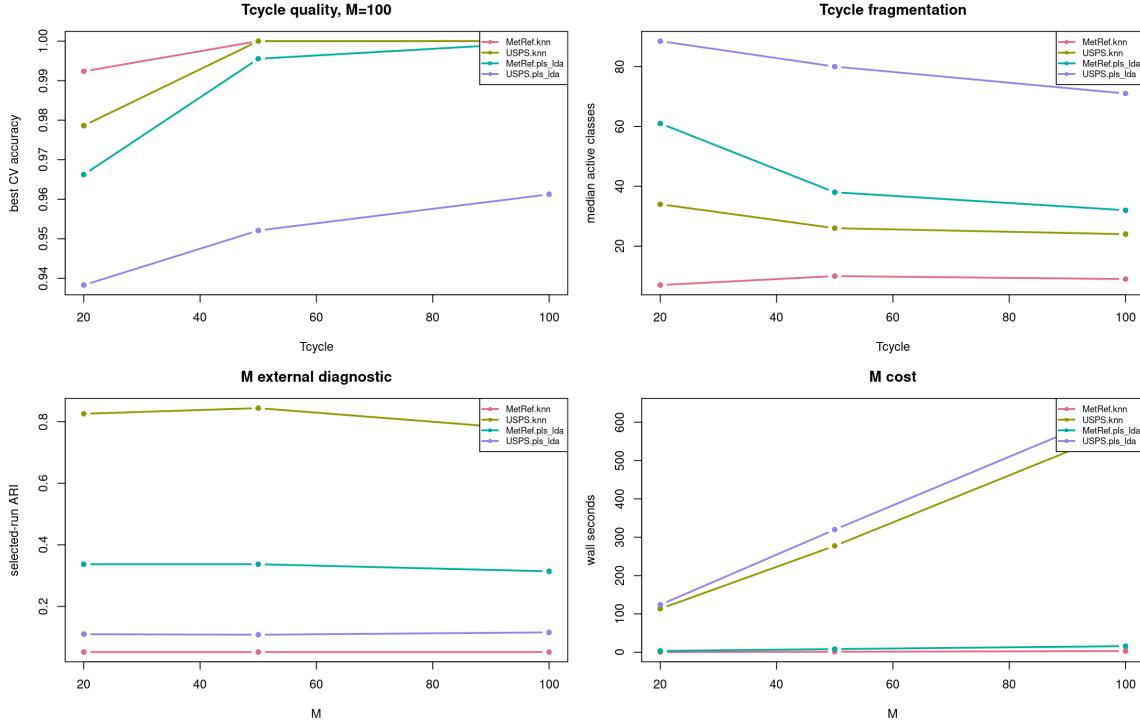


Figure 4: Sensitivity of KODAMA quality and runtime to Tcycle and M on named matrix-input datasets.

6 Availability and reproducibility

kodama-cpp is intended for release in the public tkcaccia/kodama-cpp GitHub repository. Original project code is licensed under MIT, while identified compatible third-party portions retain their upstream terms. In particular, the Metal backend is marked MIT AND Apache-2.0 because its fastEmbedR lineage retains the Faiss-mlx fused list-scan/top-k organization under Apache-2.0. The recommended project split is a standalone C++17 core repository, an R wrapper repository, and a Python wrapper repository.

Reproducibility is supported through CMake builds, CPU, CUDA, and Metal tests, wrapper validation tests, benchmark scripts, and explicit recording of backend parameters. GitHub Actions builds and tests the CPU core on Linux and macOS, builds the native Metal core on macOS, and installs and tests both wrapper packages. A separate LLVM workflow measures CPU source coverage, while Doxygen publishes the typed API and a

Table 28: Convergence of agreement-graph edge weights. RMSE entries show $M = 10 / 20 / 50$ relative to $M = 100$.

Dataset	Classifier	RMSE to M100	Correlation at M50
MetRef	KNN	0.1377 / 0.0783 / 0.0407	0.9888
MetRef	PLS-LDA	0.0588 / 0.0453 / 0.0180	0.9988
USPS	KNN	0.1011 / 0.0617 / 0.0319	0.9955
USPS	PLS-LDA	0.1191 / 0.0803 / 0.0380	0.9926

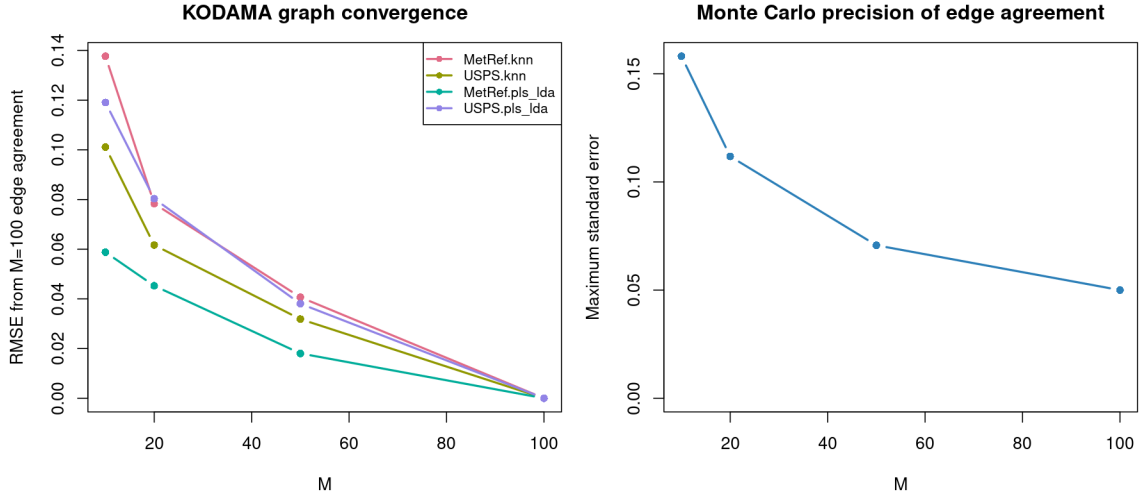


Figure 5: Convergence of edge-agreement weights as independent KODAMA runs are added. RMSE is measured against $M = 100$; the right panel shows the Bernoulli worst-case Monte Carlo standard error.

compact C++/R/Python walkthrough at <https://tkcaccia.github.io/kodama-cpp/>. The C++ core does not depend on R data readers; wrappers translate host-language objects into contiguous matrices before calling the library.

A versioned provenance audit maps every shipped source-like file to an SPDX license identifier and records pinned KODAMA, fastPLS, fastEmbedR, faissR, FAISS, Faiss-mlx, and cuVS snapshots. Stefano Cacciatore confirmed that tkcaccia is his historical Git identity and separately authorized MIT distribution of the KODAMA and fastPLS-derived portions whose copyright he owns. The audit preserves publication coauthor credit, third-party notices, and full license texts, and is enforced by a release-time source-header and retained-license checksum test.

Table 29: Implementation claims and validation evidence.

Feature claim	Where implemented	Test proving it	Benchmark proving benefit
Float32 numerical paths	MatrixView overload in include/kodama/kodama.hpp; DenseF PLS-LDA workspaces in src/plscv.cpp; CUDA and Metal device buffers remain float32.	tests/test_cv.cpp and tests/test_metal.cpp check float32 KN-NCV/PLSLDACV outputs, backend metadata, requested components, and accuracy thresholds.	The benchmark suite loads contiguous float32 matrices and records CPU, CUDA, and Metal runtime and accuracy separately.
Package-owned CPU HNSW	src/native_knn.cpp implements HNSW over contiguous float32 vectors and flat adjacency arrays. It completes a deterministic connected base seed of $\min(n, \max(n_threads + 1, 2 \cdot m_HNSW + 1))$ points before per-node-locked parallel insertion, cached reciprocal-edge distances, reusable batched candidate workspaces, and batched parallel queries. KNNCV, CoreKNN, graph construction, and KODAMA.matrix use it without linking FAISS.	tests/test_cv.cpp requires valid ordered self-excluding graphs and at least 0.99 brute-force recall for both one-thread and four-thread builds. A 100-build stress test records minimum exact recall, while dependency-free CTest covers folds, prediction accuracy, graph construction, and KODAMA optimization; the full CPU suite also passes ThreadSanitizer.	On a 4,000 by 32 Apple M3 benchmark with $k = 30$, the three-run median was 6.722 s with one thread and 1.791 s with four (3.75x scaling), versus 7.248 s for the previous four-thread path (4.05x improvement). Median one/four-thread graph overlap was 0.99966. Across 100 repeated 600 by 24 builds with $k = 20$, minimum and mean recall versus brute force were 0.99975 and 0.99987. On MetRef, native CPU KN-NCV reproduced accuracy 0.827033.
Single-owner graph lifecycle	src/kodama_matrix.cpp retains global_graph through the independent runs, compacts and optionally fuses rows in place, applies agreement correction in the same buffers, and moves the graph into KODAMAMatrixResult. The result no longer owns base_knn beside knn; R and Python serialize one graph.	tests/test_cv.cpp compares lazy in-place correction with the edge-agreement formula, verifies stable index and distance storage addresses, and checks storage accounting. Wrapper tests require one knn payload and correction-state metadata.	For the flow18 dimensions (1,000,021 samples, $k = 100$), the lifecycle benchmark reduced retained graph capacity from 2,408,050,568 to 808,016,968 bytes and maximum RSS from 2,354,304 to 791,808 KiB (66.4%). The graph ownership/copy phase fell from 0.561 to 0.059 s (89.4%); complete process wall time, including graph creation and a fixed RSS observation hold, fell from 1.60 to 1.07 s.
CUDA nearest-neighbor search	src/native_cuda_backend.cu implements package-owned float32 exact KNN, signed-hash IVF-Flat, GPU k-means, resident inverted lists, exact-pilot recall tuning, and bounded row-batched exact k-means assignment; KNNCV.CUDA and CoreKNN call it directly.	CUDA tests explicitly exercise exact and IVF index types, tuning metadata, constrained folds, float32 CoreKNN, and end-to-end KODAMAMatrix.CUDA initialization. A clean build and ldd audit exclude FAISS/cuVS/RAFT/RMM; the 44,808-sample release run activates the bounded assignment path across four lanes.	Native IVF spot checks produced 0.973857 in 4.233 s on MNIST70k and 0.816724 in 0.195 s on MetRef. Macosko2015 retina completed $M=100/Tcycle=100$ in 97.295 s for KNN and 708.579 s for PLS-LDA on a 16 GiB GPU.

Table 30: Implementation claims and validation evidence (continued).

Feature claim	Where implemented	Test proving it	Benchmark proving benefit
Resident accelerator IVF lifecycle	src/resident_ivf.cpp exposes a move-only ResidentIVFIndex. CUDA and Metal retain the float32 input, projection, centroids, list offsets, and IDs; assignment accumulation, empty-cluster repair, list counts, prefix offsets, and ID scatter execute on-device.	tests/test_cv.cpp and tests/test_metal.cpp build one resident index, run two self-searches, verify self-exclusion and metadata, and require identical indices and distances. tests/test_public_api_0.1.cpp compile-links the ownership and search surface.	On a deterministic 50,000 by 32 systems microbenchmark, 1,000-query reuse was 3.45x faster than rebuild-plus-search on CUDA and 3.86x faster on Metal, with neighbor overlap 1.000. All-row reuse was 1.12x and 1.07x because search dominated construction.
Label-aware SIM-PLS PLS-LDA	src/plscv.cpp uses label/class inputs in CPU, CUDA, and Metal float32 SIMPLS paths; accelerator implementations avoid dense one-hot responses where possible.	CPU/CUDA/Metal tests check PLSLDACV sizes, constrained folds, selected component reporting, back-end metadata, and accuracy thresholds.	At audited core commit 9da48ee, the matched MetRef M = 100, Tcycle = 100 PLS-LDA run required 341.648 s on four CPU cores and 270.408 s on CUDA, a 1.26x end-to-end speedup with identical median raw CV accuracy 0.992366; peak host-process memory was 351.4/729.7 MB. In a five-run fixed-50-component kernel check, median PLSLDACV time was 0.790 s on four Mac CPU threads and 0.202 s on Metal (3.90x), with accuracy 0.990836/0.989691. On chiamaka, corresponding kernel medians were 1.294 s on four CPU threads and 0.308 s on CUDA (4.20x), with accuracy differing by at most one sample.
Reusable fold and data buffers	PLSFoldXCacheF in src/plscv.cpp caches fold assignments, train/validation indices, scaled matrices, and CUDA Gram buffers; CoreKNN precomputes fold neighbors.	tests/test_cv.cpp checks CoreKNN/CorePLSLDA result sizes, worker/scheduler metadata, and that optimization does not decrease initial CV accuracy.	CorePLSLDA rows report 26.5x speedup on MetRef and 10.4x speedup on USPS for the CUDA core path.

Table 31: Implementation claims and validation evidence (continued).

Feature claim	Where implemented	Test proving it	Benchmark proving benefit
Exact-quota landmarking	src/kodama_matrix.cpp forms coarse matrix-derived strata, allocates exact population-proportional quotas by randomized systematic rounding, and samples rows without replacement.	tests/test_cv.cpp checks the exact effective count, distinct and represented samples, fixed-seed repeatability, and CPU/CUDA landmark-diagnostic parity.	On flow18, 750,016 landmarks were selected from 300/300 coarse matrix strata in 0.321 s; the complete M=1, Tcycle=10 CUDA KNN call took 362.348 s, of which 335.079 s was global graph construction.
Native Apple Metal backend	src/metal_backend.mm owns persistent Metal state, exact and IVF-Flat KNN, Lloyd k-means, MPS matrix products, label-aware SIMPLS, and latent-space LDA.	tests/test_metal.cpp exercises exact/IVF KNNCV, graphs, CoreKNN/CorePLSLDA, matrix KODAMA, graph-input KODAMA, and explicit rejection of unavailable Metal clustering.	MetRef exact KNN and PLS-LDA matched CPU accuracy; MNIST10k recalled IVF preserved exact accuracy while reducing KNNCV time from 2.054 to 1.662 s.
Graph-input KODAMA	The graph-input entry points in src/kodama_matrix.cpp accept supplied neighbor indices/distances; the KNN path reuses the graph and the PLS-LDA graph-only path builds self-tuning Laplacian features.	R and Python wrapper tests exercise matrix_graph with KNN and PLS-LDA, and C++ tests cover graph construction, clustering, and visualization outputs.	Graph-input is presented as an optional API. The manuscript separately states that graph-only PLS-LDA should not be interpreted as equivalent to data-input PLS-LDA without matched benchmark evidence.
Float32 randomized PCA	src/pca.cpp supplies package-owned float32 preprocessing, sketching, QR, compact eigensolve, orientation, and variance reporting; src/pca_cuda.cu and the Metal matrix-multiply adapter execute dominant products on the selected accelerator.	tests/test_cv.cpp checks dimensions, finite values, centered scores, orthonormal loadings, ordered singular values, determinism, float/double input parity, and CUDA agreement; tests/test_metal.cpp checks Metal agreement; R and Python tests exercise the public wrappers.	PCA is an auxiliary public primitive rather than part of the KODAMA objective. Performance claims will be reported separately from KODAMA.matrix timings when a matched PCA benchmark is archived.
Visualization parity and initialization	src/visualization.cpp exposes KODAMAVisualizationPCAIInit and CPU UMAP/openTSNE; the CUDA embedding kernels supply the corresponding accelerator optimizers. KODAMA.matrix builds the full-data graph once before M and, when requested, uses one native PCA to retain both scaled starts. R and Python wrappers enforce explicit/stored/raw/fallback precedence and backend matching, so supplying raw data again does not recompute an already compatible stored start.	C++ tests verify a single graph build for M > 1, both retained initialization matrices, centering, openTSNE scale, backend metadata, and raw-data overloads; wrapper tests verify default stored starts and graph-only fallbacks. A controlled CPU openTSNE comparison at the pinned fastEmbedR revision produced zero maximum and mean coordinate difference.	On flow18 (1,000,021 samples), matched CUDA KNN runs with Tcycle = 100 took 300.314 s for M = 1 and 301.091 s for M = 2; both reported graph_builds = 1, and the added independent 100-cycle search cost only 0.78 s wall time. UMAP graph construction, initialization, schedule, and objective match the pinned source. Later CPU coordinates are assessed by embedding diagnostics rather than bitwise equality because kodama-cpp is float32-native while the compared fastEmbedR R path stores iterative coordinates as doubles.

Table 32: Release-validation evidence tracked by the project.

Area	Current evidence
Correctness	CTest passed on dependency-free CPU, Apple Metal, and CUDA builds; tests cover constrained folds, predictions, confusion matrices, requested component counts, and backend parity.
Numerics	Float32 MatrixView validation tests exercise CPU, CUDA, and Metal KNN/PLS-LDA paths with typed backend metadata; a fresh CUDA configure/build verifies that SIMPLS has no Armadillo or double-fallback dependency.
Performance	Kernel-level KNNCV/PLSLDACV timings are separated from core optimizer and KODAMA.matrix timings in the benchmark outputs.
Continuous integration	GitHub Actions commit a32f718 passed the CI, CPU-coverage, and API-documentation workflows, including Linux/macOS CPU, native Metal, and wrapper jobs.
Coverage and documentation	LLVM 22.1.1 CPU instrumentation measured 63.58% line, 57.03% branch, 66.04% function, and 67.61% region coverage. CUDA and Metal are excluded from that CPU number and retain hardware tests. Doxygen generation and GitHub Pages deployment passed, and the public API site is reachable at https://tkcaccia.github.io/kodama-cpp/ .
Release	CMake install targets, wrapper build scripts, benchmark drivers, SPDX source headers, a pinned provenance matrix, retained third-party license texts, and a checksum-backed license audit are present. Commit a32f718 and R-wrapper commit 4d5506b are the current candidates; benchmark rows retain their recorded historical commits. The final clean tag, archive digest, and optional DOI will be inserted only after they are created.

6.1 Installation paths

6.2 Wrapper validation

6.3 License and dependency notes

7 Limitations

The current implementation prioritizes KNN and PLS-LDA because they are central to the KODAMA optimization principle and can be implemented cleanly on CPU, CUDA, and Apple Metal. Additional classifiers are outside this release unless they preserve the cross-validated accuracy objective and can be tested without changing the method for a specific benchmark.

The main engineering limitation is that KODAMA performs many cross-validation fits. This makes memory locality, buffer reuse, label compaction, and GPU scheduling as important as asymptotic complexity. The present release includes float32 buffers, fold reuse, label compaction, a resident full-data graph, persistent per-worker CUDA/Metal classifier workspaces, and native accelerator KNN and PLS-LDA kernels. Landmark selection and proposal control remain host orchestrated; a fully device-side evolutionary state machine is an engineering extension rather than a claim of the current manuscript.

The native CUDA neighbor-graph builder currently supports at most 256 retained neighbors per sample, whereas the CPU path supports larger rows. The implementation raises

Table 33: Installation and runtime checks for the standalone core and wrappers.

Target	Command or check
C++ CPU	Configure with CMake using <code>KODAMA_ENABLE_CUDA=OFF</code> and <code>KODAMA_ENABLE_METAL=OFF</code> ; then build, test, and install the exported target.
C++ Metal	On macOS, configure with <code>KODAMA_ENABLE_METAL=ON</code> while CUDA is off; then build and run CTest. Apple system frameworks are linked automatically.
C++ CUDA	Activate an environment containing the CUDA toolkit; configure with <code>KODAMA_ENABLE_CUDA=ON</code> ; then build and run CTest. FAISS, cuVS, RAFT, RMM, Armadillo, and external graph-clustering libraries are not required.
R wrapper	Clone https://github.com/tkcaccia/kodama-r , set <code>KODAMA_CPP_ROOT</code> and <code>KODAMA_CPP_BUILD_DIR</code> , then run <code>R CMD INSTALL kodama-r</code> . The wrapper links the installed or selected core build and includes dependency-free KODAMA-compatible normalization, scaling, and synthetic-manifold utilities.
Python wrapper	Use <code>split-repos/kodama-python</code> from the reviewed core repository and pass <code>KODAMA_CPP_ROOT</code> and <code>KODAMA_CPP_BUILD_DIR</code> as CMake config settings to pip. Publication of the separately versioned Python repository remains a pre-submission action.
Runtime check	Run <code>ctest</code> for the core library; in R call <code>KODAMA.diagnostics()</code> ; in Python import <code>kodama</code> and run the package validation tests.

an error rather than silently truncating k . This matters when comparing with interfaces that couple landmark count and graph size; the current-CRAN MetRef systems comparison therefore reports the exactly matched CPU4 path and does not manufacture an unmatched CUDA row.

8 KODAMA pseudocode

Algorithm 1 isolates one independent run inside the M-run ensemble. It starts from one landmark label vector, uses the previous CV predictions to propose label changes, evaluates each proposal with exactly one CV pass, and returns the best label vector found by that run.

Algorithm 1. One independent KODAMA run

Input:

`X_L`: landmark matrix for this run
`y0`: initial label vector on landmarks
`groups`: movable units, either samples or constrained groups
`fixed`: optional fixed-label mask
`folds`: CV fold assignment
`F`: classifier in {KNN, PLS-LDA}
`Tcycle`: proposal/evaluation cycles

Output:

`y_best`: optimized labels for this run
`pred_best`: CV predictions attached to `y_best`
`acc_best`: raw CV accuracy of `y_best`

Table 34: Local CPU/Metal and CUDA validation results for the core and wrapper packages.

Target	Validation result
C++ core, local CPU	Dependency-free release-candidate build passed 4/4 configured tests on macOS, including the source/license audit and frozen public-API snapshot.
C++ core, Apple Metal	Native Metal release-candidate build passed 5/5 configured tests, including the source/license audit and frozen public-API snapshot; a clean external CMake consumer linked the installed package and selected Metal.
C++ core, CUDA	Commit a32f718 passed 4/4 configured license, numerical/core, public-API, and float32 smoke tests in the CUDA 13.0 compute-capability-12.0 build on chiamaka.
R wrapper, local CPU	R CMD build followed by R CMD check <code>--as-cran --no-manual</code> passed on the source tarball in a UTF-8 locale with only the expected new-submission NOTE.
Python wrapper, local CPU	An isolated local package load against the CPU extension passed all six pytest checks, including explicit R/Python signature and default-parity assertions.
GitHub Actions	Commit a32f718 passed CI, CPU-coverage, and API-documentation workflows, including Ubuntu/macOS CPU, native Metal, R/Python, and documentation jobs.

```

pred0 <- CrossValidate(F, X_L, y0, folds)
acc0 <- Accuracy(y0, pred0)
score0 <- ObjectiveScore(F, y0, acc0)
y_best <- y0; pred_best <- pred0; acc_best <- acc0; score_best <- score0
y_current <- y0; pred_current <- pred0; acc_current <- acc0; score_current <- score0

for t = 1,...,Tcycle do
  if evolutionary_search then
    y_base <- y_current; pred_base <- pred_current
  else
    y_base <- y_best; pred_base <- pred_best
  end if

  y_prop <- y_base
  qmax <- ProposalBudget(t, Tcycle, number_of_groups)
  q <- UniformInteger(1, qmax)
  selected_groups <- SampleWithoutReplacement(groups, q)

  for each group g in selected_groups do
    E <- {i in members(g): fixed[i] != 1}
    if E is empty then continue
    replacement <- SampleEmpirical(pred_base[E])
    y_prop[E] <- replacement
  end for

  y_prop <- ClassTransitionProposals(y_prop, pred_base, fixed)
  if F == PLS-LDA then
    y_prop <- PLSLDATransitionCoarsening(y_prop, pred_base, fixed)
  end if

```

Table 35: Release notes for core components and external dependencies.

Component	Release note
kodama-cpp project code	MIT, with SPDX copyright assigned to Stefano Cacciatore; historical Git identity tkcaccia.
KODAMA-derived code	Public origin GPL (≥ 2); local portions whose copyright Stefano Cacciatore owns are separately authorized under MIT. KODAMA coauthors remain credited.
fastPLS-derived code	Public origin GPL-3; local SIMPLS/LDA portions whose copyright Stefano Cacciatore owns are separately authorized under MIT. fastPLS coauthors remain credited.
fastEmbedR and faissR	MIT snapshots pinned for randomized-PCA policy, UMAP/openTSNE, HNSW, Metal, and nearest-neighbor implementation lineage.
Native CPU HNSW	Direct FAISS-derived organization under MIT; Meta and Stefano Cacciatore notices and the complete FAISS license are retained.
Apple Metal	MIT AND Apache-2.0 for the combined file; Meta, Faiss-mlx, and modification notices plus both full licenses are retained.
Native CUDA	Package-owned exact/IVF KNN, k-means, and SIMPLS/LDA use CUDA Toolkit libraries without FAISS, cuVS, RAFT, RMM, Armadillo, or external graph-clustering links.
Audit enforcement	All tracked source-like files carry SPDX identifiers; official license texts are SHA-256 checked by tools/check_license_headers.sh.

```

pred_prop <- CrossValidate(F, X_L, y_prop, folds) # exactly one CV pass
acc_prop <- Accuracy(y_prop, pred_prop)
score_prop <- ObjectiveScore(F, y_prop, acc_prop)

if score_prop > score_best then
  y_best <- y_prop; pred_best <- pred_prop
  acc_best <- acc_prop; score_best <- score_prop
end if

if evolutionary_search and
  Accept(score_prop, score_current, acc_current, t, Tcycle, rng) then
  y_current <- y_prop; pred_current <- pred_prop
  acc_current <- acc_prop; score_current <- score_prop
end if

if acc_prop == 1 and (!guarded_diversity or score_prop >= score_best) then break
end for

return y_best, pred_best, acc_best

```

8.1 KODAMA.matrix ensemble

Algorithm 2 repeats Algorithm 1 independently M times, projects each optimized run back to all samples, and builds the final KODAMA-corrected graph from agreement across the optimized label vectors. The full-data neighbor graph is constructed once before the M loop; an optional single PCA supplies both stored visualization starts.

Algorithm 2. KODAMA.matrix ensemble

Input:


```

X: n by p data matrix
F: classifier in {KNN, PLS-LDA}
M: number of independent runs
Tcycle: proposal/evaluation cycles per run
landmarks, splitting, graph_neighbors
optional starting_labels, constrain, fixed

Output:
C: M by n matrix of optimized label vectors
A: length-M vector of best raw CV accuracies
G_K: KODAMA-corrected neighbor graph
Z_umap, Z_tsne: optional backend-native PCA starts

X32 <- float32(X)
L_eff <- ValidateLandmarks(landmarks, n) # ceil(0.75*n) when request >= n
GO <- KNNGraph(X32, graph_neighbors) # exactly once, before M
Z_umap, Z_tsne <- VisualizationPCAStarts(X32) # one PCA, if requested
for r = 1,...,M do
  rng <- RNG(seed + r)
  coarse <- KMeans(X32, splitting, rng)
  L <- QuotaLandmarks(coarse, L_eff, rng)
  if starting_labels supplied then
    y0 <- starting_labels[L]
  else
    y0 <- ExpressionKMeans(X32[L], splitting, rng)
  end if
  groups <- ConstrainedGroups(constrain[L])
    or one singleton group per landmark
  y0 <- ConstrainedMajority(y0, groups)
  folds <- AssignCVFolds(groups, y0)

  y_best, pred_best, acc_best <- OneIndependentRun(
    X32[L], y0, groups, fixed[L], folds, F, Tcycle, rng)

  C[r, ] <- ProjectLabelsToAllSamples(y_best, X32, L, F, groups, GO)
  A[r] <- acc_best
end for

G_K <- ReweightGraphByLabelAgreementInPlace(GO, C)
return C, A, G_K, Z_umap, Z_tsne

QuotaLandmarks(strata, L, rng, ordered):
  for each stratum c do
    q[c] <- L * size(c) / n
    take[c] <- floor(q[c])
    frac[c] <- q[c] - take[c]
  end for
  R <- L - sum_c take[c]
  u <- Uniform(0, 1)
  SystematicRound(frac, R, u, ordered, take)
  return union_c SampleWithoutReplacement(c, take[c], rng)

ObjectiveScore(F, y, a):
  score <- a
  if guarded_diversity then
    p_l <- class proportions in y
    score <- score * sqrt(1 - sum_l p_l^2)
  end if
  if F == PLS-LDA and class_coarsening then
    score <- score - (1 - a) * Parsimony(y)
  end if
  return score

Accept(s_new, s_old, a_old, t, Tcycle, rng):

```

```
if s_new >= s_old then return TRUE
tau <- max(1e-9, 0.10 * max(0, 1 - a_old) * (1 - t / Tcycle))
return Uniform(0, 1, rng) < exp((s_new - s_old) / tau)
```

8.2 KODAMA.matrix.graph from supplied neighbors

Algorithm 3 keeps the applicable classifier-specific label-evolution policy while replacing global neighbor-search construction with caller-supplied indices and distances. KNN voting, label projection, and agreement correction use the supplied graph. When the original feature matrix is absent, self-tuning Laplacian features provide selection and initialization geometry and, for PLS-LDA, the classifier representation.

Algorithm 3. KODAMA.matrix.graph from supplied neighbors

```

Input:
  I, D: n by k neighbor indices and distances
  optional X: original n by p data matrix
  F: classifier in {KNN, PLS-LDA}
  M, Tcycle, landmarks, splitting
  optional starting_labels, constrain, fixed

Output:
  C: M by n matrix of optimized label vectors
  A: length-M vector of best raw CV accuracies
  G_K: KODAMA-corrected graph on the supplied neighborhoods

GO <- NormalizeExternalGraph(I, D)
if X is supplied then
  X_work <- float32(X)
else
  X_work <- SelfTuningLaplacianFeatures(GO, ncomp)
end if
# For KNN, X_work supplies selection/initialization geometry only;
# graph voting, projection, and correction continue to use GO.

for r = 1,...,M do
  rng <- RNG(seed + r)
  L <- SelectLandmarksFromGraphOrData(GO, X_work, landmarks, rng)
  groups <- ConstrainedGroups(constrain[L])
             or one singleton group per landmark
  y0 <- InitialLabelsFromGraphOrData(GO[L], X_work[L], starting_labels[L],
                                     splitting, groups, rng)
  folds <- AssignCVFolds(groups, y0)

  if F == KNN then
    y_best, pred_best, acc_best <- Algorithm1(GO[L], y0, groups, fixed[L], folds, KNN, Tcycle)
  else
    y_best, pred_best, acc_best <- Algorithm1(X_work[L], y0, groups, fixed[L], folds, PLS-LDA, Tcycle)
  end if

  C[r, ] <- ProjectLabelsToAllSamples(y_best, X_work, L, F, groups, GO)
  A[r] <- acc_best
end for

G_K <- ReweightGraphByLabelAgreementInPlace(GO, C)
return C, A, G_K

```

Acknowledgments and Disclosure of Funding

The authors thank contributors to the KODAMA, fastPLS, and fastEmbedR software projects.

References

- Ebtesam A. Abdel-Shafy, Moussa Kassim, Alessia Vignoli, Farag Mamdouh, Svitlana Tyekucheva, Dalia Ahmed, Dupe Ojo, Brendon Price, Fabio Socciarelli, Nancy Paola Duarte-Delgado, Martin Ocharo, Chiamaka Jessica Okeke, Luca Triboli, David A. MacIntyre, Massimo Loda, Devanand Sarkar, Dinesh Gupta, Silvano Piazza, Luiz Fernando Zerbini, Leonardo Tenori, and Stefano Cacciatore. Kodama enables self-guided weakly supervised learning in spatial transcriptomics. *bioRxiv*, 2025. doi: 10.1101/2025.05.28.656544. Preprint.
- Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002.
- Asa Ben-Hur, Andre Elisseeff, and Isabelle Guyon. A stability based method for discovering structure in clustered data. In *Pacific Symposium on Biocomputing*, pages 6–17, 2002.
- Stefano Cacciatore and Leonardo Tenori. *KODAMA: Knowledge Discovery by Accuracy Maximization*, 2026. R package documentation, CRAN.
- Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.
- Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.
- Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien. *Semi-Supervised Learning*. MIT Press, 2006.
- Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemo-metrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.
- Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2: 193–218, 1985.
- Jeff Johnson, Matthijs Douze, and Herve Jegou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019.
- Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pages 1137–1143, 1995.
- Nikolaus Kriegeskorte, W. Kyle Simmons, Patrick S. F. Bellgowan, and Chris I. Baker. Circular analysis in systems neuroscience: the dangers of double dipping. *Nature Neuroscience*, 12:535–540, 2009.
- Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, 2013.

- George C. Linderman, Manas Rachh, Jeremy G. Hoskins, Stefan Steinerberger, and Yuval Kluger. Fast interpolation-based t-sne for improved visualization of single-cell rna-seq data. *Nature Methods*, 16:243–245, 2019.
- Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d’Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.
- Alexander Ratner, Christopher De Sa, Sen Wu, Daniel Selsam, and Christopher Ré. Data programming: Creating large training sets, quickly. In *Advances in Neural Information Processing Systems*, 2016.
- Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
- Sören Sonnenburg, Mikio L. Braun, Cheng Soon Ong, Samy Bengio, Leon Bottou, Geoffrey Holmes, Yann LeCun, Klaus-Robert Müller, Fernando Pereira, Carl Edward Rasmussen, Gunnar Rätsch, Bernhard Schölkopf, Alexander Smola, Pascal Vincent, Jason Weston, and Robert C. Williamson. The need for open source software in machine learning. *Journal of Machine Learning Research*, 8:2443–2466, 2007.
- Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems*, 2003.
- Xiaojin Zhu, Zoubin Ghahramani, and John Lafferty. Semi-supervised learning using gaussian fields and harmonic functions. In *Proceedings of the Twentieth International Conference on Machine Learning*, pages 912–919, 2003.